

PHY 4830: Computational Astrophysics and Geophysics

Condensed summary of lectures

Dipankar Bhattacharya

Department of Physics, Ashoka University

Preliminaries

- The majority of physical problems handled in Astrophysics and Geophysics pertain to material and energy transport.
- Material can be in the form of solid, liquid or gas; incompressible or compressible
- Physics of mass movement would include Gravity, Pressure, Magnetic Field, Viscosity...and thermodynamics.
- Energy transport may occur via conduction, convection, radiation
- Energy production and loss to the surroundings play an important role in both material and energy movement
- Energy production may occur by Radioactivity, Chemical Reactions, nuclear fusion, accretion process etc

All these processes are expressed through, in general, Partial Differential Equations.

Example: Fluid motion

$$\frac{\partial \rho}{\partial t} + \vec{\nabla} \cdot (\rho \vec{v}) = 0 \quad [\text{continuity}] \quad (1)$$

$$\rho \left[\frac{\partial \vec{v}}{\partial t} + (\vec{v} \cdot \vec{\nabla}) \vec{v} \right] + \vec{\nabla} P - \rho \vec{g} - \vec{\nabla} \cdot \{\tau\} = 0 \quad [\text{momentum}] \quad (2)$$

$$\frac{\partial}{\partial t} \left(\frac{1}{2} \rho v^2 + \rho u \right) + \vec{\nabla} \cdot \left[\left(\frac{1}{2} \rho v^2 + \rho h \right) \vec{v} \right] = 0 \quad [\text{energy}] \quad (3)$$

where ρ =density, $\vec{v}$ =fluid velocity, P =pressure, $\vec{g}$ =acceleration due to gravity, $\{\tau\}$ =deviatorial stress tensor, u =internal energy per unit mass and h =enthalpy per unit mass.

Magnetic Field:

$$\frac{\partial \vec{B}}{\partial t} - \vec{\nabla} \times (\vec{v} \times \vec{B}) + \vec{\nabla} \times (\eta \vec{\nabla} \times \vec{B}) = 0 \quad [\text{induction}] \quad (4)$$

$$\rho \left[\frac{\partial \vec{v}}{\partial t} + (\vec{v} \cdot \vec{\nabla}) \vec{v} \right] + \vec{\nabla} P - \rho \vec{g} - \vec{\nabla} \cdot \{\tau\} - \frac{\vec{J} \times \vec{B}}{c} = 0 \quad [\text{momentum}] \quad (5)$$

where $\vec{B}$ =magnetic field, $\vec{J}$ =current density, η =magnetic diffusivity= $\frac{c^2}{4\pi\sigma}$

Radiative Transfer:

$$\frac{dI_\nu}{d\tau_\nu} = -I_\nu + S_\nu \quad (6)$$

where I_ν =Specific Intensity, τ_ν =optical depth, S_ν =source function; all at frequency ν

Thermal and Radiative Energy Transport:

$$\vec{F} = -[k_{\text{rad}} + k_{\text{cd}} + k_{\text{conv}}]\vec{\nabla}T \quad (7)$$

where k_{rad} , k_{cd} and k_{conv} are effective conductivities due to radiation, conduction and convection respectively.

Numerical solution will involve the use of digital computers

Algorithms typically require the problem to be cast as a set of coupled first order equations. For example, Newtonian equation of motion

$$\frac{d^2\vec{r}}{dt^2} = \vec{F}(\vec{r}, \vec{r}) \quad (8)$$

may be written as a system of coupled first order ODEs:

$$\frac{dx}{dt} = v_x \quad (9)$$

$$\frac{dy}{dt} = v_y \quad (10)$$

$$\frac{dz}{dt} = v_z \quad (11)$$

$$\frac{dv_x}{dt} = F_x(x, y, z, v_x, v_y, v_z) \quad (12)$$

$$\frac{dv_y}{dt} = F_y(x, y, z, v_x, v_y, v_z) \quad (13)$$

$$\frac{dv_z}{dt} = F_z(x, y, z, v_x, v_y, v_z) \quad (14)$$

So in general one would solve a system of ODEs of the form

$$\frac{dy_i(x)}{dx} = f_i(x, y_1, \dots, y_n); \quad i = 1, \dots, N \quad (15)$$

with Boundary Conditions - two kinds : Initial Value and Boundary Value problems.

Basic things to keep in mind while using digital computers

- All numbers are represented in binary with a finite number of bits.
- Within available range, Integers can be represented exactly, not all floats can.
- Addition/subtraction of floats of different magnitude cause significant loss of precision.
- Two types of error to remain aware of and control:
 1. **Truncation Error** caused by taking finite number of steps in an algorithm while infinite number are required
 2. **Roundoff Error** caused by finite number of bits in number representation where infinite number are required

Floating point representation in a digital computer is typically implemented by specifying 3 quantities: a sign s , a biased exponent e and a mantissa M . The represented value being

$$x = s \times M \times 2^{e-E}$$

where E is the exponent bias. Two versions of these are usually available - a 32-bit single precision (*float*) and a 64-bit double precision (*double*). The IEEE 754 standard implementation specifies for single precision the arrangement of 1 bit sign, 8 bit biased exponent and 23-bit mantissa while for double precision this becomes 1 bit sign, 11 bit biased exponent and 52 bit mantissa.

The bits of the mantissa are binary coefficients of negative powers of two. The most significant bit is thus the coefficient of $\frac{1}{2}$, the next lower bit the coefficient of $\frac{1}{4}$, etc. For any representable number, the exponent can always be adjusted to make the most significant bit of the mantissa equal to 1. This is called the *normalised* form of the mantissa. Since the first bit of the normalised mantissa is always 1, it in fact need not be stored, giving one extra effective bit for the mantissa. Numbers for which this is not possible are considered to be outside the representable set of floating point numbers in the implementation - these include infinities, NaNs and underflows. Zero cannot be normalised either, and it is represented by the special pattern of all exponent and mantissa bits being set to zero.

Initial Value Problems in Ordinary Differential Equations

Discretization of ODEs (Differencing):

$$\frac{dy}{dx} = f(x, y) \quad (16)$$

Basic Euler:

Stepsize h , steps $1, 2, 3, \dots, n-1, n, n+1, \dots, N$ between the required limits

$$y_{n+1} = y_n + hy'_n \quad (\text{Explicit}) \quad (17)$$

$$y_{n+1} = y_n + hy'_{n+1} \quad (\text{Implicit}) \quad (18)$$

where y' denotes the derivative dy/dx .

Implicit scheme requires one to solve, in general, complex nonlinear equations at every step. Explicit scheme is straightforward but less stable.

Explicit Euler is 1st order accurate (i.e. the error term is of 2nd order in h). Methods of higher order could be used to improve accuracy while keeping stepsize not too small.

Explicit vs Implicit differencing and stability:

Consider the first order linear ODE

$$y' = -cy \quad (19)$$

where $c > 0$ is a constant. This equation has a known exponentially decaying solution.

Now consider explicit differencing

$$y_{n+1} = y_n + hy'_n = (1 - ch)y_n \quad (20)$$

This gives y_{n+1} explicitly in terms of y_n , and is easy to evaluate. However the method is not stable for arbitrary choice of h . If $h > 2/c$ then the magnitude of y_n grows with every step in an oscillatory manner, whereas the real solution is a smoothly decaying one.

On the other hand, in implicit differencing

$$y_{n+1} = y_n + hy'_{n+1} = y_n - chy_{n+1} \quad (21)$$

gives

$$y_{n+1} = \frac{y_n}{1 + ch} \quad (22)$$

which is absolutely stable. For large h this converges to the correct solution (but is less accurate in the intermediate steps).

For a general non-linear derivative function, however, evaluating an implicit step can be very complex. An approximate approach in such cases is to Taylor expand the function to linear order and evaluate y_{n+1} in a manner similar to that in eq. (22):

$$y' = f(y) \quad (23)$$

$$y_{n+1} = y_n + hf(y_{n+1}) \quad (24)$$

$$y_{n+1} = y_n + h \left[f(y_n) + \left. \frac{df}{dy} \right|_{y_n} (y_{n+1} - y_n) \right] \quad (25)$$

This is called a semi-implicit method. Stability here is not guaranteed, but is often achieved.

Higher order methods

$$k_1 = hf(x_n, y_n) \quad (26)$$

$$k_2 = hf(x_n + \frac{1}{2}h, y_n + \frac{1}{2}k_1) \quad (27)$$

$$y_{n+1} = y_n + k_2 + O(h^3) \quad (28)$$

This is second order accurate. This method is called the *Midpoint method*, and is also known as the *Runge-Kutta method of second order*.

This technique can be extended to higher orders, adjusting coefficients to cancel errors of desired orders. For example the popular *Fourth order Runge Kutta method* takes the following steps:

$$k_1 = hf(x_n, y_n) \quad (29)$$

$$k_2 = hf(x_n + \frac{h}{2}, y_n + \frac{k_1}{2}) \quad (30)$$

$$k_3 = hf(x_n + \frac{h}{2}, y_n + \frac{k_2}{2}) \quad (31)$$

$$k_4 = hf(x_n + h, y_n + k_3) \quad (32)$$

$$y_{n+1} = y_n + \frac{k_1}{6} + \frac{k_2}{3} + \frac{k_3}{3} + \frac{k_4}{6} + O(h^5) \quad (33)$$

which is fourth order accurate.

One may solve a step using a trial h , then again using two steps of $h/2$ each. If $y^{(1)}$ and $y^{(2)}$ are the corresponding values of computed y , then $\Delta = |y^{(1)} - y^{(2)}|$ is an estimate of the accuracy achieved. If the desired accuracy is Δ_0 then the stepsize can be adjusted to

$$h_{\text{new}} = \left(\frac{\Delta_0}{\Delta}\right)^{1/5} h \quad (34)$$

since the error term is of order h^5 . This is called adaptive stepsize control.

Runge-Kutta Method can be extended to even higher orders. The number of function evaluations per order gain is however the best for the Fourth order method.

Modified Midpoint Method

The Midpoint method discussed earlier can be extended to the following. Consider the ODE $y' = f(x, y)$. At a given value of x , $y(x)$ has been evaluated. We then consider the next integration step to be to $x + H$, where H is a relatively large interval. We begin by subdividing the interval H into n parts: $h = H/n$. The step from x to $x + H$ is then constructed as follows.

$$z_0 \equiv y(x) \quad (35)$$

$$z_1 = z_0 + hf(x, z_0) \quad (36)$$

$\vdots$

$$z_{m+1} = z_{m-1} + 2hf(x + mh, z_m) \quad (37)$$

$\vdots$

$$y(x + H) \approx y_n \equiv \frac{1}{2} [z_n + z_{n-1} + hf(x + H, z_n)] \quad (38)$$

The error in this evaluation contains only even powers of h :

$$|y_n - y(x + H)| = \sum_{i=1}^{\infty} \alpha_i h^{2i} \quad (39)$$

Combination of intermediate evaluations to construct a higher order method will thus gain two orders, each time an error term is eliminated. For example,

$$y(x + H) = \frac{4y_{[h]} - y_{[2h]}}{3} \quad (40)$$

is fourth order accurate. Here $y_{[h]}$ and $y_{[2h]}$ are values of $y(x + H)$ obtained using n and $n/2$ steps respectively. Note that the latter does not involve any additional function evaluation, it can be constructed just from the alternate terms of $y_{[h]}$. The quantity $|y_{[h]} - y_{[2h]}|$ can also be used to provide an error estimate of the integration step, and h or H can be suitably adjusted based on the same.

Bulirsch-Stoer method

This involves the following steps:

1. Take a large Eulerian step H and evaluate $y_H = y(x + H)$.
2. Now subdivide the step into $h = H/k$, and again evaluate y_H with steps of size h , for example by the Modified Midpoint method (note that other methods can also be used).
3. Do this for multiple times, $i = 1, \dots, N$, with typically $k = 2^i$
4. Stop when the difference between successive evaluations of y_H falls below a pre-set threshold.
5. Fit a polynomial or rational function (ratio of polynomials) through the evaluated values of y_H as a function of h . Extrapolate the fit to $h = 0$ (Richardson Extrapolation). If the error terms contain only even orders of h then the extrapolation could be done as a function of h^2 , leading to quicker convergence.

This method works well and provides high accuracy for smooth derivative functions $f(x, y)$. If the function instead has sharp jumps, discontinuities or is a set of tabulated values, then Runge-Kutta would be the method of choice.

Conservative second order ODE

We have mentioned that a higher order differential equation needs to be broken up into a set of simultaneous first order equations before differencing. While that is true in general, for conservative second order equations of the form

$$y'' = f(x, y) \tag{41}$$

where the first order derivative is absent, direct differencing is possible, leading to about a factor of two saving in computational cost.

Let $y(x_0) = y_0$ and $y'(x_0) = z_0$ be the specified boundary conditions. The differencing may then be done using *Stoermer's Rule*. As above, we will deal with a large step H and its subdivisions h :

$$h = H/n \quad (42)$$

$$y_1 = y_0 + h \left[z_0 + \frac{1}{2} h f(x_0, y_0) \right] \quad (43)$$

$$y_{m+1} - 2y_m + y_{m-1} = h^2 f(x_0 + mh, y_m), \quad m = 1, \dots, n-1 \quad (44)$$

$$z_n = (y_n - y_{n-1})/h + \frac{1}{2} h f(x_0 + H, y_n) \quad (45)$$

y_n and z_n are then used as initial values for the next interval H of the integration.

Direct implementation of the algorithm in the above form could be susceptible to significant roundoff errors. Accumulating just the differences Δ between steps alleviates this to some extent (Henrici implementation):

Let $\Delta_0 \equiv h[z_0 + hf(x_0, y_0)/2]$ and $y_1 = y_0 + \Delta_0$. Then for $m = 1 \dots n-1$

$$\begin{aligned} \Delta_m &= \Delta_{m-1} + h^2 f(x_0 + mh, y_0) \\ y_{m+1} &= y_m + \Delta_m \end{aligned}$$

And

$$z_n = \frac{\Delta_{n-1}}{h} + \frac{1}{2} h f(x_0 + H, y_n)$$

This method can also be subjected to Richardson Extrapolation if desired.

Predictor-Corrector Methods:

This is a general multi-step implicit method. Earlier we have discussed one-step implicit methods, where y_{n+1} depends only on one previous step y_n . In general, however, the derivative can be approximated as a polynomial passing through several previous points $x_n, x_{n-1}, \dots$ and possibly even through x_{n+1} . In that case,

$$y_{n+1} = y_n + h(\beta_0 y'_{n+1} + \beta_1 y'_n + \beta_2 y'_{n-1} + \beta_3 y'_{n-2} + \dots) \quad (46)$$

The method is explicit if $\beta_0 = 0$, and is implicit otherwise. In the predictor-corrector implementation, first an approximate prediction of y_{n+1} is made using an explicit method, then this value is used to interpolate the derivative and correct the estimate of y_{n+1} . The difference between the predictor and the corrector estimates gives a measure of the truncation error and can be

used to control the stepsize. Because of the use of function evaluations at multiple previous steps, this is also referred to as a *Multistep* method. An optimized implementation with third order accuracy uses

$$\text{Adams-Bushford Predictor: } y_{n+1} = y_n + \frac{h}{12}(23y'_n - 16y'_{n-1} + 5y'_{n-2}) + O(h^4)$$

$$\text{Adams-Moulton Corrector: } y_{n+1} = y_n + \frac{h}{12}(5y'_{n+1} + 8y'_n - y'_{n-1}) + O(h^4) \quad (47)$$

A variant of the above is a *Multivalue* method, where higher order derivatives evaluated at a single step is made use of. For example, consider the differential equation

$$y'(x) = f(x, y)$$

Let us define a vector

$$\vec{y}_n = \begin{pmatrix} y_n \\ hy'_n \\ \frac{h^2}{2}y''_n \\ \frac{h^3}{6}y'''_n \end{pmatrix}$$

One may then write for any x close to x_n

$$y(x) = y_n + (x - x_n)y'_n + \frac{(x - x_n)^2}{2}y''_n + \frac{(x - x_n)^3}{6}y'''_n$$

in which the values of the components of $\vec{y}$ can be substituted.

The Predictor y_{n+1} can be obtained by setting $x = x_{n+1}$. Let us denote this as $\vec{y}_{n+1}^{(p)}$:

$$\vec{y}_{n+1}^{(p)} = [B] \cdot \vec{y}_n$$

where

$$[B] = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The actual approximation to $\vec{y}_{n+1}$ in this method will then be written as

$$\vec{y}_{n+1} = \vec{y}_{n+1}^{(p)} + \alpha \vec{r}$$

in which we set

$$r_2 = 1; \quad \alpha = hf(x_{n+1}, y_{n+1}) - hy'_{n+1}^{(p)}$$

The other components of $\vec{r}$ can be chosen freely to optimise the algorithm. Different choices will give different order of the method and different stability properties. For example, $\vec{r} = (0, 1, 3/4, 1/6)$ is equivalent to the Adams-Bushford step, while $\vec{r} = (5/12, 1, 3/4, 1/6)$ is equivalent to Adams-Moulton.

Predictor-corrector methods are in extensive use in the computation of stellar evolution, where a sequence of stellar models are computed with gradually changing values of composition, luminosity etc.

Guide to solving Barotropic Stellar Structure equations

$$\frac{dP}{dr} = -\frac{GM(r)\rho(r)}{r^2} \quad (48)$$

$$\frac{dM(r)}{dr} = 4\pi r^2 \rho(r) \quad (49)$$

$$P = P(\rho) \quad [\text{Equation of State}] \quad (50)$$

- Can be solved as an initial value problem, starting at the centre.
- $r = 0$ is a singular point, so regularisation is needed.
- Note that by symmetry, derivatives must go to zero at $r = 0$, so very near the centre, ρ can be considered constant.
- Supply a value of central density $\rho(r = 0) = \rho_0$, compute the corresponding pressure P_0 from the EoS.
- Take a small increment in radius, ϵ , within which the density and pressure will be considered constant.
- $M(\epsilon)$ is now $4\pi\rho_0\epsilon^3/3$. Use P_0 and $M(\epsilon)$ as initial values at $r = \epsilon$ and integrate outwards.
- Stop integration when Pressure falls to zero. This gives $r = R$, the radius of the star, and $M(R) = M$, the mass of the star.

Boundary Value Problems in Ordinary Differential Equations

Unlike initial value problems, here all the boundary conditions are not specified at one end, so forward integration from a defined state is not possible.

Let us consider two-point boundary value problems for an ODE of the form

$$\frac{d\vec{y}}{dx} = \vec{g}(x, \vec{y}) \quad (51)$$

where $\vec{y}$ represents the array of N dependent variables $y_1, \dots, y_N$, and g_i are the corresponding derivatives dy_i/dx .

Boundary conditions are specified at $x = x_1$ and $x = x_2$ in the following manner:

$$B_{1j}(x_1, \vec{y}(x_1)) = 0, \quad j = 1 \dots n_1 \quad (52)$$

$$B_{2k}(x_2, \vec{y}(x_2)) = 0, \quad k = 1 \dots n_2; \quad n_1 + n_2 = N \quad (53)$$

Shooting Method:

This involves starting at one end, with guess values of unspecified variables, and forward integrate like an initial value problem. One may integrate all the way to the other end, or starting from both ends to an intermediate fitting point. At the end of this, one evaluates the mismatch between the expected and the estimated values and corrects the initial guesses to eliminate this difference.

Shooting to the other end:

- Start integration at $x = x_1$
- Use boundary conditions to specify n_1 variables at x_1
- For remaining $n_2 = N - n_1$ variables specify a random n_2 -component vector $\vec{V}$, whose components may stand for these missing values or for parameters from which those missing values may be derived. We then have

$$y_i(x_1) = y_i(x_1; V_1, \dots, V_{n_2}), \quad i = 1 \dots N \quad (54)$$

The components of $\vec{V}$ are the unknowns to be solved for.

- With the above starting values, use the ODE to integrate to x_2 and evaluate $\vec{y}(x_2)$.

- Now define a discrepancy vector $\vec{F} = \vec{B}_2(x_2, \vec{y})$, which has n_2 components. For the correct solution, the components of $\vec{F}$ should be all zero,
- One then proceeds to estimate a correction vector $\delta\vec{V}$ which, when added to $\vec{V}$, renders $\vec{F}$ to null.
- To estimate $\delta\vec{V}$, it is common to employ the Newton's method, which amounts to solving the system of linear equations

$$[\mathbf{J}] \cdot \delta\vec{V} = -\vec{F} \quad (55)$$

where the Jacobian matrix $[\mathbf{J}]$ consists of elements $J_{ij} = \partial F_i / \partial V_j$

- To evaluate each element J_{ij} a separate integration of the N ODEs is required.

$$\frac{\partial F_i}{\partial V_j} = \frac{F_i(V_1, \dots, V_j + \Delta V_j, \dots) - F_i(V_1, \dots, V_j, \dots)}{\Delta V_j} \quad (56)$$

- Iterate the above to improve the solution

Shooting to a fitting point:

- Start with a n_2 -element guess vector $\vec{V}^{(1)}$ at x_1 and a n_1 -element guess vector $\vec{V}^{(2)}$ at x_2 . This now has a total of N unknowns.
- Integrate from both ends to an intermediate fitting point x_f .
- The difference between the estimates $\vec{y}^{(1)}(x_f)$ and $\vec{y}^{(2)}(x_f)$ defines an N -element discrepancy vector
- Use Newton's method to solve for N corrections to the elements of the $\vec{V}$ -s, and iterate to improve.

Relaxation methods for boundary value problems in ODEs:

As in the case of shooting method, we consider two-point boundary value problems for an ODE of the form

$$\frac{d\vec{y}}{dx} = \vec{g}(x, \vec{y}) \quad (57)$$

where $\vec{y}$ represents the array of N dependent variables $y_1, \dots, y_N$, and g_i is the corresponding derivative dy_i/dx .

We convert this to a finite difference equation, defining mesh points x_k , $k = 1, \dots, M$. Values of $\vec{y}$ are now defined only at the mesh points, $\vec{y}_k$, $k = 1, \dots, M$. Our goal is now to determine the $M \times N$ values of the dependent variables, using the Finite Difference Equation (FDE)

$$\vec{y}_k - \vec{y}_{k-1} - (x_k - x_{k-1})\vec{g}\left[\frac{1}{2}(x_k + x_{k-1}), \frac{1}{2}(\vec{y}_k + \vec{y}_{k-1})\right] = 0, \quad k = 2, \dots, M \quad (58)$$

which provides $(M - 1) \times N$ equations, and the N boundary conditions

$$\vec{B}(x_1, \vec{y}_1) = 0; \quad \vec{C}(x_M, \vec{y}_M) = 0 \quad (59)$$

where $\vec{B}$ has n_1 non-zero components and $\vec{C}$ has $n_2 = N - n_1$ non-zero components. One may arrange the order of the dependent variables such that $\vec{B}$ has its first n_2 components zero and $\vec{C}$ has its last n_1 components zero.

The relaxation method starts with assigning arbitrary, or “guess” values of $\vec{y}_k$ at all k , then iteratively improves the solution.

At each stage, one computes the following error vectors:

$$\begin{aligned} \vec{E}_k &= \vec{y}_k - \vec{y}_{k-1} - (x_k - x_{k-1})\vec{g}\left[\frac{1}{2}(x_k + x_{k-1}), \frac{1}{2}(\vec{y}_k + \vec{y}_{k-1})\right]; k = 2, \dots, M \\ \vec{E}_1 &= \vec{B}(x_1, \vec{y}_1) \\ \vec{E}_{M+1} &= \vec{C}(x_M, \vec{y}_M) \end{aligned} \quad (60)$$

The aim now is to render all the components of these error vectors to zero simultaneously.

One usually employs Newton’s method to accomplish this.

In first order Taylor expansion, $\vec{E}_k(\vec{y}_k + \Delta\vec{y}_k, \vec{y}_{k-1} + \Delta\vec{y}_{k-1})$ may be written as

$$\vec{E}_k(\vec{y}_k, \vec{y}_{k-1}) + \sum_{n=1}^N \frac{\partial \vec{E}_k}{\partial y_{n,k-1}} \Delta y_{n,k-1} + \sum_{n=1}^N \frac{\partial \vec{E}_k}{\partial y_{n,k}} \Delta y_{n,k} \quad (61)$$

Which can then be set to zero to obtain, component-wise,

$$\sum_{n=1}^N S_{j,n}^{(k)} \Delta y_{n,k-1} + \sum_{n=N+1}^{2N} S_{j,n}^{(k)} \Delta y_{n-N,k} = -E_{j,k}; \quad j = 1 \dots N, k = 2 \dots M \quad (62)$$

Where $S_{j,n}$ has been defined as a $N \times 2N$ Jacobian matrix at each k with the first N set of elements in the second index corresponding to derivatives w.r.t. $y_{n,k-1}$ and the next N set of elements to derivatives w.r.t. $y_{n,k}$.

These are to be augmented at the boundaries with

$$\sum_{n=1}^N S_{j,n}^{(1)} \Delta y_{n,1} = -E_{j,1}, \quad j = n_2 + 1, \dots, N \quad (63)$$

$$\sum_{n=1}^N S_{j,n}^{(M+1)} \Delta y_{n,M} = -E_{j,M+1}, \quad j = 1, \dots, n_2 \quad (64)$$

The matrix elements $S_{j,n}^{(k)}$ can be evaluated by differentiating the expression of the FDE.

This now is a system of linear algebraic equations to solve for. One derives the values of $\Delta \vec{y}_k$, updates $\vec{y}_k$ and repeats the process until the magnitude of the corrections fall below a pre-set threshold. The iterative approach is necessary as in general the errors are not linear functions of the corrections and hence the first order Taylor expansion used in Newton's method is an approximation.

Partial Differential Equations

Consider the example of a linear homogeneous first order Partial Differential Equation

$$a \frac{\partial \phi(x, y)}{\partial x} + b \frac{\partial \phi(x, y)}{\partial y} = 0 \quad (65)$$

Introduce a change of variables

$$s = ax + by; \quad t = bx - ay \quad (66)$$

Then

$$\frac{\partial \phi}{\partial x} = \frac{\partial \phi}{\partial s} \frac{\partial s}{\partial x} + \frac{\partial \phi}{\partial t} \frac{\partial t}{\partial x} = a \frac{\partial \phi}{\partial s} + b \frac{\partial \phi}{\partial t} \quad (67)$$

and

$$\frac{\partial \phi}{\partial y} = \frac{\partial \phi}{\partial s} \frac{\partial s}{\partial y} + \frac{\partial \phi}{\partial t} \frac{\partial t}{\partial y} = b \frac{\partial \phi}{\partial s} - a \frac{\partial \phi}{\partial t} \quad (68)$$

So the PDE becomes

$$(a^2 + b^2) \frac{\partial \phi}{\partial s} = 0 \quad (69)$$

Which means that for a given t , the function ϕ is independent of s . Therefore the ϕ -value is constant on t =constant lines. These lines are called *characteristics*.

Now if boundary conditions are specified on a curve on xy plane, defining ϕ at each point along the curve, then where a characteristic intersects this line, the value of ϕ at the intersection point is propagated along the characteristic. Different characteristic lines carry such values from different intersection points, thus defining the solution of the PDE over the entire space.

This then suggests some constraints on how the boundary conditions need to be specified:

- The curve on which boundary conditions are specified should not be coincident with a characteristic
- In case of multiple intersections of a boundary curve with a characteristic, the specification must be consistent at all the intersection points

Partial Differential Equations of second order

A large variety of physical processes are described by Partial Differential Equations (PDEs) of second order, which we will delve into in some detail.

Some examples of commonly encountered PDEs in Physics:

$$\begin{aligned}
 \nabla^2 \psi &= 0 && \text{Laplace's Equation} \\
 \nabla^2 \psi &= 4\pi\rho && \text{Poisson's Equation} \\
 \nabla^2 \psi + k^2 \psi &= 0 && \text{Wave Equation} \\
 \nabla^2 \psi &= \frac{1}{a^2} \frac{\partial \psi}{\partial t} && \text{Diffusion Equation} \\
 -\frac{\hbar^2}{2m} \nabla^2 \psi + V\psi &= i\hbar \frac{\partial \psi}{\partial t} && \text{Schrödinger's Equation}
 \end{aligned}$$

We will develop our discussions mainly through PDEs in two independent variables, but the techniques can be easily generalised to multiple dimensions. Let us consider a second order linear differential operator

$$\mathcal{L}(x, y) \equiv a \frac{\partial^2}{\partial x^2} + b \frac{\partial^2}{\partial x \partial y} + c \frac{\partial^2}{\partial y^2} + d \frac{\partial}{\partial x} + e \frac{\partial}{\partial y} + f(x, y) \quad (70)$$

An equation of the form $\mathcal{L}\psi = 0$ would be called *homogeneous* and one like $\mathcal{L}\psi = g(x, y)$ would be called *inhomogeneous*. The Poisson's equation is an example of the latter.

Such equations are usually classified according to the shape of their characteristic curves:

$$\begin{aligned}
 \text{Hyperbolic} \quad &\text{if } ac - b^2 < 0 \quad \text{e.g. } \frac{\partial^2 u}{\partial t^2} = v^2 \frac{\partial^2 u}{\partial x^2} \text{ (Wave equation)} \\
 \text{Parabolic} \quad &\text{if } ac - b^2 = 0 \quad \text{e.g. } \frac{\partial u}{\partial t} = \frac{\partial}{\partial x} \left(D \frac{\partial u}{\partial x} \right) \text{ (Diffusion equation)} \\
 \text{Elliptic} \quad &\text{if } ac - b^2 > 0 \quad \text{e.g. } \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = \rho(x, y) \text{ (Poisson's equation)}
 \end{aligned}$$

For numerical implementation the more relevant classification is into initial value and boundary value problems. For example, the first two equations above constitute Initial Value Problems and the third is an example of a Boundary Value Problem.

A common approach to the numerical solution of a Partial Differential Equation is Finite Differencing, The function values are defined on a chosen

grid defined by discrete values of the independent variables. In case of Initial value problems, the solution starts from a row of fully defined values, proceeding to compute subsequent rows one at a time, taking into account the boundary conditions specified at the two ends of each row. In this method, only a few rows need to be stored at any given time.

In boundary value problems, the specified boundary conditions are spread across multiple rows. One then needs to solve for values over the entire mesh simultaneously, thus all rows require to be maintained in memory at any given time.

Flux Conservative Initial Value Problems in PDEs:

This refers to PDEs of the type

$$\frac{\partial u}{\partial t} = -\frac{\partial F}{\partial x} \quad (71)$$

Different classes of PDEs can be reduced to this form. For example the 1-d wave equation

$$\frac{\partial^2 \psi}{\partial t^2} = v^2 \frac{\partial^2 \psi}{\partial x^2} \quad (72)$$

may be written as

$$\frac{\partial r}{\partial t} = v \frac{\partial s}{\partial x}; \quad \frac{\partial s}{\partial t} = v \frac{\partial r}{\partial x} \quad (73)$$

where

$$r = v \frac{\partial \psi}{\partial x}; \quad s = \frac{\partial \psi}{\partial t} \quad (74)$$

Considering (r, s) as two components of a vector $\vec{u}$, the flux can be defined as

$$\vec{F}(\vec{u}) = \begin{pmatrix} 0 & -v \\ -v & 0 \end{pmatrix} \cdot \vec{u} \quad (75)$$

For illustration, let us consider a prototypical case where the flux is defined as $vu(x, t)$ with $v = \text{constant}$. The PDE then has the form

$$\frac{\partial u}{\partial t} = -v \frac{\partial u}{\partial x} \quad (76)$$

This has a known solution of the form $u = f(x - vt)$.

The differencing would begin by considering equally spaced grid points

$$x_j = x_0 + j\Delta x, \quad j = 0, 1, 2, \dots, J$$

$$t_n = t_0 + n\Delta t, \quad n = 0, 1, 2, \dots, N$$

We write u_j^n as representing $u(t_n, x_j)$.

Then we replace the time derivative $(\partial u / \partial t)_{j,n}$ with a forward Euler difference

$$\frac{u_j^{n+1} - u_j^n}{\Delta t}$$

which is first order accurate.

As the values of u at all spatial grid points are available at any corresponding instant of time, we can replace the spatial derivative $(\partial u / \partial x)_{j,n}$ with a second order accurate difference that is centred in space:

$$\frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x}$$

Combining, we have the FDE:

$$\frac{u_j^{n+1} - u_j^n}{\Delta t} = -v \left(\frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} \right) \quad (77)$$

or

$$u_j^{n+1} = u_j^n - \frac{v\Delta t}{2\Delta x} (u_{j+1}^n - u_{j-1}^n) \quad (78)$$

This is called a **Forward Time Centred Space (FTCS)** scheme which is explicit, simple to derive and implement, and it gives j -th element of u at a timestep $n + 1$ in terms of three elements ($j - 1, j, j + 1$) of u at the previous timestep n .

However, this scheme is prone to instability. This behaviour can be revealed through von Neumann stability analysis, where a solution of the difference equation is sought in the form

$$u_j^n = \xi_{\mathcal{K}}^n \exp(i\mathcal{K}j\Delta x) \quad (79)$$

where the wavenumber $\mathcal{K}$ indicates the inverse spatial scale of oscillation of the Fourier component, and ξ is an amplification factor by which the amplitude of the solution, and of any departure from it, gets effectively multiplied at each time step.

Substituting this in the difference equation, one finds

$$\xi_{\mathcal{K}} = 1 - i \frac{v\Delta t}{\Delta x} \sin(\mathcal{K}\Delta x) \quad (80)$$

i.e. $|\xi| > 1$ for all $\mathcal{K}$, indicating the inherent instability.

Lax Method:

A simple modification of the FTCS differencing scheme avoids the above instability – replacing the u_j^n term on the rhs of eq. (78) by $(u_{j+1}^n + u_{j-1}^n)/2$. This is called the LAX method:

$$u_j^{n+1} = \frac{1}{2}(u_{j+1}^n + u_{j-1}^n) - \frac{v\Delta t}{2\Delta x}(u_{j+1}^n - u_{j-1}^n) \quad (81)$$

Here the von Neumann amplification factor works out to be

$$\xi_{\mathcal{K}} = \cos(\mathcal{K}\Delta x) - i\frac{v\Delta t}{\Delta x} \sin(\mathcal{K}\Delta x) \quad (82)$$

giving $|\xi_{\mathcal{K}}|^2 \leq 1$ for

$$\frac{v\Delta t}{\Delta x} \leq 1 \quad (83)$$

The choice of mesh spacing and timestep must always obey eq. (83) to ensure stability. This is known as the **Courant condition**.

The Lax method happens to be equivalent to adding a diffusion term to the original PDE. The difference equation (81) can be rewritten as

$$\frac{u_j^{n+1} - u_j^n}{\Delta t} = -v \left(\frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} \right) + \frac{1}{2} \left(\frac{u_{j+1}^n - 2u_j^n + u_{j-1}^n}{\Delta t} \right) \quad (84)$$

which is the finite difference form of the PDE

$$\frac{\partial u}{\partial t} = -v \frac{\partial u}{\partial x} + \frac{(\Delta x)^2}{2\Delta t} \frac{\partial^2 u}{\partial x^2} \quad (85)$$

The diffusion term (called numerical viscosity) causes modes, particularly of small scale ($\mathcal{K}\Delta x \sim 1$), to dissipate quickly, stabilising the scheme. This is acceptable as long as one is not interested in the behaviour of the small scale modes in the solution.

Other errors that may be of relevance include phase errors and transport errors.

Phase errors: The amplification factor can be rewritten as

$$\xi_{\mathcal{K}} = \exp(-i\mathcal{K}\Delta x) + i \left(1 - \frac{v\Delta t}{\Delta x} \right) \sin \mathcal{K}\Delta x$$

The first term gives the correct phase relationship for a propagating wave. However unless $v\Delta t$ is exactly equal to Δx , the second term contributes phase errors that could be significant at small scales (large $\mathcal{K} \sim 1/\Delta x$).

Transport errors: In purely advective problems, the dependent variable is transported at flow velocity in a certain direction. In the example above, u is transported at speed v . So at a timestep $n + 1$, u_j^{n+1} should depend on previous u values at mesh points from which advection could occur to the mesh point j . Depending on the direction of v , these could be points at lower or higher values of mesh index, but never both. But in the differencing schemes used above, u_j^{n+1} depends both on u_{j-1}^n and u_{j+1}^n , which is contrary to physical expectation. In such cases it is appropriate to adopt *upwind differencing*, where

$$\frac{u_j^{n+1} - u_j^n}{\Delta t} = -v_j^n \begin{cases} \frac{u_j^n - u_{j-1}^n}{\Delta x}, & v_j^n > 0 \\ \frac{u_{j+1}^n - u_j^n}{\Delta x}, & v_j^n < 0 \end{cases} \quad (86)$$

This, though, loses the second order accuracy in space. Interpolation schemes may be used to achieve higher order accuracy in this case.

Second order accuracy in time:

In order to achieve second order accuracy in time, one may difference an advective PDE

$$\frac{\partial u}{\partial t} = -\frac{\partial F}{\partial x}$$

as

$$u_j^{n+1} - u_j^{n-1} = -\frac{\Delta t}{\Delta x} (F_{j+1}^n - F_{j-1}^n) \quad (87)$$

For constant v this would be

$$u_j^{n+1} - u_j^{n-1} = -\frac{v\Delta t}{\Delta x} (u_{j+1}^n - u_{j-1}^n) \quad (88)$$

This is known as the **Staggered Leapfrog method**. von-Neumann stability analysis of this yields

$$\xi_{\mathcal{K}} = -i \frac{v\Delta t}{\Delta x} \sin(\mathcal{K}\Delta x) \pm \sqrt{1 - \left(\frac{v\Delta t}{\Delta x} \sin(\mathcal{K}\Delta x) \right)^2} \quad (89)$$

So $|\xi_{\mathcal{K}}| = 1$ (stable) when $v\Delta t \leq \Delta x$ (Courant condition).

This method connects mesh points in a chessboard pattern – the “white” and the “black” squares remain disconnected to each other. Errors can then build up between them, which is called *mesh drifting*. To control this a small numerical viscosity may be added to eq. (88):

$$u_j^{n+1} - u_j^{n-1} = -\frac{v\Delta t}{\Delta x}(u_{j+1}^n - u_{j-1}^n) + \epsilon(u_{j+1}^n - 2u_j^n + u_{j-1}^n) \quad (90)$$

where $\epsilon \ll 1$.

Lax-Wendroff method is another implementation that avoids large numerical viscosity and mesh drifting. This is a two step method, in which first a provisional half step is created:

$$u_{j+\frac{1}{2}}^{n+\frac{1}{2}} = \frac{1}{2}(u_{j+1}^n + u_j^n) - \frac{\Delta t}{2\Delta x}(F_{j+1}^n - F_j^n) \quad (91)$$

Using this value of u , the flux $F_{j+\frac{1}{2}}^{n+\frac{1}{2}}$ is evaluated.

Finally, these half-step F -s are used to obtain

$$u_j^{n+1} = u_j^n - \frac{\Delta t}{\Delta x} \left(F_{j+\frac{1}{2}}^{n+\frac{1}{2}} - F_{j-\frac{1}{2}}^{n+\frac{1}{2}} \right) \quad (92)$$

The provisional half step values may then be discarded.

Extension to multiple dimensions

Consider a problem of the type

$$\frac{\partial u}{\partial t} = - \left(\frac{\partial F_x}{\partial x} + \frac{\partial F_y}{\partial y} \right) \quad (93)$$

One introduces a spatial grid

$$x_j = x_0 + j\Delta; \quad y_l = y_0 + l\Delta$$

and carries out the Lax differencing as

$$\begin{aligned} u_{j,l}^{n+1} &= \frac{1}{4}(u_{j+1,l}^n + u_{j-1,l}^n + u_{j,l+1}^n + u_{j,l-1}^n) \\ &\quad - \frac{\Delta t}{2\Delta}(F_{x,j+1,l}^n - F_{x,j-1,l}^n + F_{y,j,l+1}^n - F_{y,j,l-1}^n) \end{aligned} \quad (94)$$

Writing $F_x = v_x u$ and $F_y = v_y u$, von Neumann stability analysis gives the stability condition as

$$\Delta t \leq \frac{\Delta}{\sqrt{2(v_x^2 + v_y^2)}}$$

In general, for N dimensions, the Courant condition works out to be

$$\Delta t \leq \frac{\Delta}{\sqrt{N}|v|} \quad (95)$$

Diffusive initial value problems:

Consider equations of the type

$$\frac{\partial u}{\partial t} = \frac{\partial}{\partial x} \left(D \frac{\partial u}{\partial x} \right) \quad (96)$$

This can be cast in the form of flux conservative equations by writing

$$F = -D \frac{\partial u}{\partial x} \quad (97)$$

so that

$$\frac{\partial u}{\partial t} = -\frac{\partial F}{\partial x} \quad (98)$$

Methods described for flux conservative equations can be adapted to solve this.

However, there are more effective methods that can be constructed by direct differencing of the second order derivative.

To illustrate, we consider cases where D is a constant.

One of the simplest schemes is FTCS:

$$\frac{u_j^{n+1} - u_j^n}{\Delta t} = D \left[\frac{u_{j+1}^n - 2u_j^n + u_{j-1}^n}{(\Delta x)^2} \right] \quad (99)$$

If D is not a constant then the FTCS differencing may take the following form:

$$\frac{u_j^{n+1} - u_j^n}{\Delta t} = \frac{1}{2(\Delta x)^2} \left[(D_{j+1}^n + D_j^n)u_{j+1}^n - (D_{j+1}^n + 2D_j^n + D_{j-1}^n)u_j^n + (D_j^n + D_{j-1}^n)u_{j-1}^n \right]$$

Unlike the first order case, the FTCS scheme here is stable, with the stability criterion

$$\frac{2D\Delta t}{(\Delta x)^2} \leq 1 \quad (100)$$

But this demands too small a time step to reasonably cover diffusion on spatial scales of many grid points, as the diffusion time scales as square of the length scale.

Practical implementation therefore requires schemes that would allow much larger time steps. One possibility is a fully implicit scheme:

$$\frac{u_j^{n+1} - u_j^n}{\Delta t} = D \left[\frac{u_{j+1}^{n+1} - 2u_j^{n+1} + u_{j-1}^{n+1}}{(\Delta x)^2} \right] \quad (101)$$

which is unconditionally stable for large time steps, at the cost of accuracy in intermediate steps.

A scheme that combines the stability of the implicit scheme with the accuracy of the explicit scheme is the **Crank-Nicolson method**, which averages the two:

$$\frac{u_j^{n+1} - u_j^n}{\Delta t} = \frac{D}{2} \left[\frac{(u_{j+1}^{n+1} - 2u_j^{n+1} + u_{j-1}^{n+1}) + (u_{j+1}^n - 2u_j^n + u_{j-1}^n)}{(\Delta x)^2} \right] \quad (102)$$

This method happens to be stable for any step size Δt , and is generally recommended for diffusive problems.

Multidimensional problems:

For problems with constant diffusivity, e.g.:

$$\frac{\partial u}{\partial t} = D \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) \quad (103)$$

We attempt generalisation of the Crank-Nicolson method:

$$u_{j,l}^{n+1} = u_{j,l}^n + \frac{1}{2} \alpha \left(\delta_x^2 u_{j,l}^{n+1} + \delta_x^2 u_{j,l}^n + \delta_y^2 u_{j,l}^{n+1} + \delta_y^2 u_{j,l}^n \right) \quad (104)$$

where

$$\alpha \equiv \frac{D\Delta t}{\Delta^2}$$

and

$$\delta_x^2 u_{j,l}^n \equiv u_{j+1,l}^n - 2u_{j,l}^n + u_{j-1,l}^n,$$

likewise δ_y^2 . This could be solved as a set of coupled linear equations.

Another way to generalise the Crank-Nicolson scheme is the **Alternating Direction Implicit (ADI)** method where one introduces two half timesteps, in each a different dimension is treated implicitly:

$$u_{j,l}^{n+1/2} = u_{j,l}^n + \frac{1}{2}\alpha \left(\delta_x^2 u_{j,l}^{n+1/2} + \delta_y^2 u_{j,l}^n \right) \quad (105)$$

$$u_{j,l}^{n+1} = u_{j,l}^{n+1/2} + \frac{1}{2}\alpha \left(\delta_x^2 u_{j,l}^{n+1/2} + \delta_y^2 u_{j,l}^{n+1} \right) \quad (106)$$

This is derived from a general method called Operator Splitting where if the differential operator $\mathcal{L}$ may be written as a sum of multiple operators

$$\mathcal{L} = \mathcal{L}_1 + \mathcal{L}_2 + \cdots + \mathcal{L}_M$$

and a differencing scheme is known for each operator

$$\begin{aligned} u^{n+1} &= \mathcal{A}_1(u^n, \Delta t) \quad \text{for } \mathcal{L}_1 \\ u^{n+1} &= \mathcal{A}_2(u^n, \Delta t) \quad \text{for } \mathcal{L}_2 \\ &\vdots \end{aligned} \quad (107)$$

$$u^{n+1} = \mathcal{A}_M(u^n, \Delta t) \quad \text{for } \mathcal{L}_M \quad (108)$$

Then a timestep may be split into M parts and one operator at a time be applied in sequence through the fractional time steps:

$$\begin{aligned} u^{n+(1/M)} &= \mathcal{A}_1(u^n, \Delta t) \\ u^{n+(2/M)} &= \mathcal{A}_2(u^{n+(1/M)}, \Delta t) \\ &\vdots \end{aligned} \quad (109)$$

$$u^{n+1} = \mathcal{A}_M(u^{n+(M-1)/M}, \Delta t) \quad (110)$$

An example problem in Astronomy where the methods just discussed can be applied is that of ohmic evolution of crustal magnetic fields of neutron stars in the presence of mass accretion. The evolution of the magnetic field in Magneto Hydro Dynamic (MHD) limit is described by the induction equation:

$$\frac{\partial \vec{B}}{\partial t} = \vec{\nabla} \times (\vec{v} \times \vec{B}) - \frac{c^2}{4\pi} \vec{\nabla} \times \left(\frac{1}{\sigma} \vec{\nabla} \times \vec{B} \right) \quad [\text{in cgs units}]$$

We will consider the neutron star and the accretion flow on it to be spherically symmetric. The density ρ inside the star and the electrical conductivity σ are functions of the star-centred radial coordinate r , and $r = R$ denotes the stellar surface.

Accretion of matter at a rate $\dot{M}$ causes the material of the neutron star to move inwards with the speed

$$v(r) = -\frac{\dot{M}}{4\pi r^2 \rho(r)}$$

For simplicity, let us consider the external magnetic field to be a pure dipole. The vector potential would then have only an azimuthal component: $\vec{A} = (0, 0, A_\phi)$; the field $\vec{B} = \vec{\nabla} \times \vec{A}$. For a dipole field $A_\phi = g(r, t) \sin \theta / r$. The induction equation then becomes

$$\frac{\partial g(r, t)}{\partial t} = v(r) \frac{\partial g(r, t)}{\partial r} + \frac{c^2}{4\pi\sigma} \left[\frac{\partial^2 g(r, t)}{\partial r^2} - \frac{2g(r, t)}{r^2} \right]$$

With the boundary conditions

$$\begin{aligned} \frac{\partial g}{\partial r} + \frac{g}{R} &= 0 \quad \text{at } r = R \\ g(r, t) &= 0 \quad \text{at } r = r_c \end{aligned}$$

where r_c is the inner boundary of the crust.

The RHS is composed of an advective and a diffusive term. This can be solved by operator splitting, treating the advective term via upwind differencing and the diffusive term by the Crank-Nicolson scheme modified to suit the radial dependence of conductivity.

(e.g. Konar, S, Bhattacharya, D. 1997, Monthly Notices of the Royal Astronomical Society, 284, 311)

Boundary value problems:

Consider an elliptic equation of the form

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = \rho(x, y) \quad (111)$$

with appropriate boundary conditions.

We introduce a 2-D mesh

$$\begin{aligned} x &= x_0 + j\Delta, \quad j = 0, 1, \dots, J \\ y &= y_0 + l\Delta, \quad l = 0, 1, \dots, L \end{aligned}$$

and write the finite difference equation as

$$\frac{u_{j+1,l} - 2u_{j,l} + u_{j-1,l}}{\Delta^2} + \frac{u_{j,l+1} - 2u_{j,l} + u_{j,l-1}}{\Delta^2} = \rho_{j,l} \quad (112)$$

i.e.

$$u_{j+1,l} + u_{j-1,l} + u_{j,l+1} + u_{j,l-1} - 4u_{j,l} = \Delta^2 \rho_{j,l} \quad (113)$$

This can be written as a set of linear equations if u is unwrapped into a 1-D array and written as a vector u_i where $i = j(L + 1) + l$. Then

$$u_{i+(L+1)} + u_{i-(L+1)} + u_{i+1} + u_{i-1} - 4u_i = \Delta^2 \rho_i \quad (114)$$

Boundary conditions are provided at $j = 0, J$ and $l = 0, L$, i.e. at

$$\begin{aligned} i &= 0, 1, \dots, L \\ i &= J(L + 1), \dots, J(L + 1) + L \\ i &= 0, L + 1, \dots, J(L + 1) \\ i &= L, (L + 1) + L, \dots, J(L + 1) + L \end{aligned} \quad (115)$$

We now take all the known quantities in eq. (114) to the rhs and express the system of equations in a matrix form:

$$[A] \cdot \vec{u} = \vec{b} \quad (116)$$

This can be solved via direct matrix inversion. However the storage requirement could be prohibitive when the mesh size $[(J + 1)(L + 1)]$ is large. In such cases Fourier methods or relaxation methods may be adopted.

Fourier Transform Method:

This is applicable when the coefficients of the derivatives are constant in space.

Fourier Transforms (FTs) can reduce differential equations to algebraic ones.

In numerical implementation we use discrete Fourier Transforms:

$$u_{jl} = \frac{1}{JL} \sum_{m=0}^{J-1} \sum_{n=0}^{L-1} \hat{u}_{mn} e^{-2\pi i jm/J} e^{-2\pi iln/L} \quad (117)$$

$$\rho_{jl} = \frac{1}{JL} \sum_{m=0}^{J-1} \sum_{n=0}^{L-1} \hat{\rho}_{mn} e^{-2\pi i jm/J} e^{-2\pi iln/L} \quad (118)$$

where the mesh size is $(J+1) \times (L+1)$, the active solution space dimension is $J \times L$ ($0 \leq j < J$; $0 \leq l < L$), the rest of the mesh being populated with *replicated* boundary conditions. The transforms are defined as:

$$\hat{u}_{mn} = \sum_{j=0}^{J-1} \sum_{l=0}^{L-1} u_{jl} e^{2\pi i jm/J} e^{2\pi iln/L} \quad (119)$$

$$\hat{\rho}_{mn} = \sum_{j=0}^{J-1} \sum_{l=0}^{L-1} \rho_{jl} e^{2\pi i jm/J} e^{2\pi iln/L} \quad (120)$$

with periodic boundary conditions:

$$u_{j,l} = u_{j+J,l} = u_{j,l+L} \quad (121)$$

Substituting expression (117) in the difference equations,

$$\hat{u}_{mn} \left(e^{2\pi im/J} + e^{-2\pi im/J} + e^{2\pi in/L} + e^{-2\pi in/L} - 4 \right) = \hat{\rho}_{mn} \Delta^2 \quad (122)$$

Here $\hat{\rho}_{mn}$ can be evaluated from the given source distribution ρ_{jl} using eq. (120). One can then compute

$$\hat{u}_{mn} = \frac{\hat{\rho}_{mn} \Delta^2}{2 \left(\cos \frac{2\pi m}{J} + \cos \frac{2\pi n}{L} - 2 \right)} \quad (123)$$

and from here evaluate the solution u_{jl} using eq. (117).

If instead of periodic boundary conditions one is specified Dirichlet boundary conditions with u going to zero at the edges of the domain, then one can use the same technique, but using Sine transforms:

$$u_{jl} = \frac{2}{J} \frac{2}{L} \sum_{m=1}^{J-1} \sum_{n=1}^{L-1} \hat{u}_{mn} \sin \frac{\pi jm}{J} \sin \frac{\pi ln}{L} \quad (124)$$

and

$$\hat{\rho}_{mn} = \sum_{j=1}^{J-1} \sum_{l=1}^{L-1} \rho_{jl} \sin \frac{\pi jm}{J} \sin \frac{\pi ln}{L} \quad (125)$$

with

$$\hat{u}_{mn} = \frac{\hat{\rho}_{mn} \Delta^2}{2 \left(\cos \frac{\pi m}{J} + \cos \frac{\pi n}{L} - 2 \right)} \quad (126)$$

For Neumann boundary conditions specifying zero derivatives at the edges of the domain, Cosine transforms may be used instead.

Note that all the transforms mentioned above can be evaluated using the Fast Fourier Transform (FFT) technique.

Relaxation methods for boundary value problems:

Consider an elliptic equation of the form $\mathcal{L}u = \rho$

Let us write the following differential equation based on it:

$$\frac{\partial u}{\partial t} = \mathcal{L}u - \rho \quad (127)$$

and solve it as a diffusive initial value problem.

As in the nature of diffusive problems, the solution reaches *equilibrium* as $t \rightarrow \infty$, where the time derivative vanishes.

The solution of the original differential equation is then the same as that of the initial value problem for large t .

For example, let us consider

$$\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} - \rho \quad (128)$$

We difference this using FTCS:

$$u_{j,l}^{n+1} = u_{j,l}^n + \frac{\Delta t}{\Delta^2} \left(u_{j+1,l}^n + u_{j-1,l}^n + u_{j,l+1}^n + u_{j,l-1}^n - 4u_{j,l}^n \right) - \rho_{j,l} \Delta t \quad (129)$$

where Δ is the spatial grid spacing in both x and y .

The stability of FTCS scheme requires that in 2-D, the time step should obey $\Delta t \leq \Delta^2/4$.

Using the largest allowed time step ($= \Delta^2/4$) in eq. (129), one finds

$$u_{j,l}^{n+1} = \frac{1}{4} \left(u_{j+1,l}^n + u_{j-1,l}^n + u_{j,l+1}^n + u_{j,l-1}^n \right) - \frac{\Delta^2}{4} \rho_{j,l} \quad (130)$$

Apart from the source term, this just means replacing a value of u with the average of that at the four nearest grid points. One may iterate this to convergence. This is called the **Jacobi Method**.

A slight modification of this is the **Gauss-Seidel Method**, where the RHS of eq. (130) is changed to

$$u_{j,l}^{n+1} = \frac{1}{4} \left(u_{j+1,l}^n + u_{j-1,l}^{n+1} + u_{j,l+1}^n + u_{j,l-1}^{n+1} \right) - \frac{\Delta^2}{4} \rho_{j,l} \quad (131)$$

This can be computed by “in-place updates”, where the values in the spatial grid are updated from step n to step $n + 1$, starting from the lowest (j, l) and ascending, such that when $u_{j,l}$ needs to be updated, the new values of $u_{j-1,l}$ and $u_{j,l-1}$ are already available.

These FDEs can be written in a matrix form

$$[A] \cdot \vec{x} = \vec{b} \quad (132)$$

and $[A]$ can be split into

$$[A] = [L] + [D] + [U] \quad (133)$$

Here $[D]$ is diagonal, while $[L]$ and $[U]$ are lower and upper diagonal respectively, without the diagonal line.

At the r -th stage of the iterations, the Jacobi method leads to linear equations of the form

$$[D] \cdot \vec{x}^{(r)} = -([L] + [U]) \cdot \vec{x}^{(r-1)} + \vec{b} \quad (134)$$

So $-[D]^{-1}([L] + [U])$ becomes the *iteration matrix* that maps one iteration of $\vec{x}$ to the next.

The convergence of this method is characterised by the rate at which undesirable residuals $(\vec{x} - \vec{x}^{(r)})$ are suppressed at each iteration. Each component of the residual vector is multiplied by a factor between 0 and 1 at each

iteration. The largest of these represents the slowest improvement at that stage and hence determines the overall rate of convergence. This factor is called the *spectral radius* ρ_s ($0 < \rho_s < 1$).

The number of iterations r required to reduce the overall error by a factor 10^{-p} can then be estimated as

$$\rho_s^r = 10^{-p} \quad (135)$$

or

$$r = \frac{p \ln 10}{(-\ln \rho_s)} \quad (136)$$

In Jacobi method, for homogeneous Dirichlet or Neumann Boundary Conditions, the spectral radius works out to be

$$\rho_{\text{Jacobi}} = \frac{\cos \frac{\pi}{J} + \left(\frac{\Delta x}{\Delta y}\right)^2 \cos \frac{\pi}{L}}{1 + \left(\frac{\Delta x}{\Delta y}\right)^2} \quad (137)$$

for a $J \times L$ grid in x, y with spacings Δx and Δy respectively.

For $\Delta x = \Delta y$, $J \times J$ grid and large J ,

$$\rho_J \approx 1 - \frac{\pi^2}{2J^2} \quad (138)$$

and corresponding

$$r \approx \frac{2p \ln 10}{\pi^2} J^2 \approx \frac{1}{2} p J^2 \quad (139)$$

which is quadratic in J , and the convergence is too slow for the method to be of practical use.

For Gauss-Seidel method, in the limit of large J ,

$$\rho_J \approx 1 - \frac{\pi^2}{J^2} \quad (140)$$

and

$$r \approx \frac{1}{4} p J^2 \quad (141)$$

which is only a factor of two smaller than that of the Jacobi method.

A more usable variant of this technique is the **Successive Over-relaxation (SOR)** method, which can be described as follows.

For Gauss-Seidel method, the equivalent of eq. (134) is

$$([L] + [D]) \cdot \vec{x}^{(r)} = -[U] \cdot \vec{x}^{(r-1)} + \vec{b} \quad (142)$$

so

$$\vec{x}^{(r)} = -([L] + [D])^{-1}([U] \cdot \vec{x}^{(r-1)} - \vec{b}) \quad (143)$$

Adding and subtracting $\vec{x}^{(r-1)}$ in the RHS,

$$\vec{x}^{(r)} = \vec{x}^{(r-1)} - ([L] + [D])^{-1} \{([L] + [D] + [U]) \cdot \vec{x}^{(r-1)} - \vec{b}\} \quad (144)$$

The quantity in the curly braces is the residual vector $\vec{\xi}^{(r-1)}$, the magnitude of which is sought to be reduced in subsequent relaxation steps. The Gauss-Seidel method thus leads to the iterations

$$\vec{x}^{(r)} = \vec{x}^{(r-1)} - ([L] + [D])^{-1} \vec{\xi}^{(r-1)} \quad (145)$$

for which, we have seen that the convergence rate is slow.

In the SOR method, the above is modified to

$$\vec{x}^{(r)} = \vec{x}^{(r-1)} - \omega([L] + [D])^{-1} \vec{\xi}^{(r-1)} \quad (146)$$

where ω is the over-relaxation parameter. $0 < \omega < 1$ leads to under-relaxation, and $\omega > 2$ produces non-convergent iterations. So for useful over-relaxation, $1 < \omega < 2$.

The optimal choice of ω happens to be

$$\omega = \frac{2}{1 + \sqrt{1 - \rho_{\text{Jacobi}}^2}} \quad (147)$$

in the limit of large number of iterations. For this choice, the spectral radius of SOR is

$$\rho_s = \left(\frac{\rho_{\text{Jacobi}}}{1 + \sqrt{1 - \rho_{\text{Jacobi}}^2}} \right)^2 \quad (148)$$

Using the asymptotic estimate of ρ_{Jacobi} from eq. (140),

$$\omega_{\text{optimal}} \approx \frac{2}{1 + \frac{\pi}{J}} \quad (149)$$

for which

$$\rho_{\text{SOR}} \approx 1 - \frac{2\pi}{J} \quad (150)$$

and

$$r \approx \frac{p \ln 10}{2\pi} J \quad (151)$$

which is proportional to J and hence much faster than the Jacobi or Gauss-Seidel method.

The implementation of SOR would proceed as follows.

Let the finite difference equation for the original PDE $\mathcal{L}u = \rho$ be written as

$$a_{j,l}u_{j+1,l} + b_{j,l}u_{j-1,l} + c_{j,l}u_{j,l+1} + d_{j,l}u_{j,l-1} + e_{j,l}u_{j,l} - f_{j,l} = 0 \quad (152)$$

At a given stage of iteration, the values of u in the mesh, if inserted into the above FDE, would yield the residuals

$$\xi_{j,l} = a_{j,l}u_{j+1,l} + b_{j,l}u_{j-1,l} + c_{j,l}u_{j,l+1} + d_{j,l}u_{j,l-1} + e_{j,l}u_{j,l} - f_{j,l} \quad (153)$$

which will be non-zero. In the next iteration, the task is to replace the $u_{j,l}$ values (which we will call $u_{j,l}^{\text{old}}$), by updated values $u_{j,l}^{\text{new}}$, such that the new residuals reduce in magnitude.

Solving eq. (152) for $u_{j,l}$, one may write

$$u_{j,l}^* = \frac{1}{e_{j,l}}(f_{j,l} - a_{j,l}u_{j+1,l} - b_{j,l}u_{j-1,l} - c_{j,l}u_{j,l+1} - d_{j,l}u_{j,l-1}) \quad (154)$$

where the values of $u_{j,l}^{\text{old}}$ are used in the RHS. $u_{j,l}^*$ are then used to compute $u_{j,l}^{\text{new}}$ via over-relaxation:

$$u_{j,l}^{\text{new}} = \omega u_{j,l}^* + (1 - \omega)u_{j,l}^{\text{old}} \quad (155)$$

This is equivalent to writing

$$u_{j,l}^{\text{new}} = u_{j,l}^{\text{old}} - \omega \frac{\xi_{j,l}}{e_{j,l}} \quad (156)$$

where $\xi_{j,l}$ is the residual in eq. (153), evaluated for $u_{j,l} = u_{j,l}^{\text{old}}$. The iterations are to be terminated when $|\vec{\xi}|$ falls below a chosen threshold.

One may notice from eq. (154) that updates to even points involve values only at odd points and vice versa. One may then accomplish this in two half-sweeps, once updating the odd points and then updating the even points. The choice of ω in eq. (147) is optimal in the asymptotic limit, but not necessarily during the initial steps. A better choice is to use *Chebyshev acceleration* at each half-step:

$$\begin{aligned}
\omega^{(0)} &= 1 \\
\omega^{(1/2)} &= 1/(1 - \rho_{\text{jacobi}}^2/2) \\
&\vdots \\
\omega^{(n+1/2)} &= 1/(1 - \rho_{\text{jacobi}}^2 \omega^{(n)}/4), \quad n = \frac{1}{2}, 1, \dots, \infty \\
&\vdots \\
\omega^{(\infty)} &\rightarrow \omega_{\text{optimal}}
\end{aligned}$$

Multi-grid methods:

Consider an elliptic equation of the form $\mathcal{L}u = f$

In finite differencing, we discretize on a mesh scale h and obtain a system of linear equations

$$\mathcal{L}_h u_h = f_h \quad (157)$$

In principle, direct matrix solution of this equation will lead to the desired result. However, as we have discussed, this is difficult for problems with large number of grid points.

Multi-grid methods attempt to address this issue by coarse graining the problem to a level at which a solution by direct matrix method is possible, then proceeding to finer grids and reducing the residuals.

Let $\tilde{u}_h$ be a trial solution at the fine grid of scale h . The *residual* or *defect* at this scale is then

$$d_h = \mathcal{L}_h \tilde{u}_h - f_h \quad (158)$$

If $v_h = u_h - \tilde{u}_h$ is the required correction then

$$\mathcal{L}_h v_h = -d_h \quad (159)$$

If this is too large a problem to solve by matrix method then one forms an approximation on a coarser grid of spacing H , and write

$$\mathcal{L}_H v_H = -d_H \quad (160)$$

The defect d_H is obtained by a coarse graining operator called a *restriction operator*

$$d_H = \mathcal{R}d_h \quad (161)$$

If $\tilde{v}_H$ is the solution to eq. (160) then one projects it back to the finer scale using a *prolongation operator*

$$\tilde{v}_h = \mathcal{P}\tilde{v}_H \quad (162)$$

and then $\tilde{u}_h$ is updated:

$$\tilde{u}_h^{\text{new}} = \tilde{u}_h + \tilde{v}_h \quad (163)$$

One may note at this point that relaxation schemes like Jacobi or Gauss-Seidel are efficient in quickly damping small scale components of the residuals but they take a very long time to damp large scale components. Solution after coarse graining, on the other hand, damps the larger scale components. So it is useful to combine both. One may thus run a few (Gauss-Seidel) relaxation steps (called *smoothing*) at the fine scale before coarse graining the problem, and do so again after mapping the problem back from coarse to fine scale.

In practice, one usually adopts coarse graining by a factor of two, namely $H = 2h$. However this may not be sufficient to afford an exact matrix solution at the coarse level. In that case some smoothing steps are performed and then the problem is coarse grained further. One can thus adopt multiple nested coarse graining levels until an exact solution becomes possible. One can then trace back through the levels to the original fine scale, applying a smoothing at each step. One full excursion like this is called a *cycle*. The steps just described would constitute what is called a “V-cycle”. However, in some cases it may be beneficial to climb up only a part of the way, perform smoothing and go back down to the coarsest level for another exact solution before climbing back all the way to the finest scale. Such a sequence is termed a “W-cycle”. Figure 1 diagrammatically displays the procedure.

In factor-of-two grid scaling, the commonly used form of the prolongation operator for a 2-D mesh can be represented as

$$\begin{bmatrix} \frac{1}{4} & \frac{1}{2} & \frac{1}{4} \\ \frac{1}{2} & 1 & \frac{1}{2} \\ \frac{1}{4} & \frac{1}{2} & \frac{1}{4} \end{bmatrix} \quad (164)$$

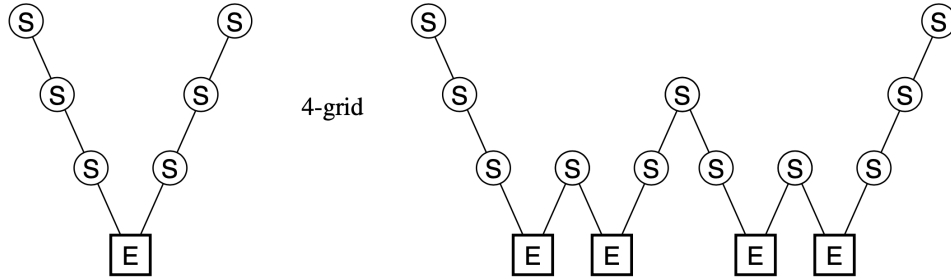


Figure 1: Illustration of multi-grid method. Finest grids are at the top and progressively coarser grids are below. E denotes the grid at which an exact solution is evaluated and S stands for smoothing steps. A V-cycle is shown on the left and a W-cycle on the right. In each cycle, action proceeds from the left to the right. Figure taken from *Numerical Recipes*, 3rd Ed, by W.H. Press *et al*, 2007, Cambridge University Press.

In this prolongation, each grid point at coarser level is surrounded by eight new grid points at the factor of two finer level. The numbers above represent the weight with which a quantity on coarse grid is multiplied and added to the new grid points. The central point is common to both grids and gets a weight 1. New points immediately left, right, up and down have two neighbouring coarse grid points, and the average of them are used to populate these points. Points on the diagonal share four neighbours on coarse grid and are populated by an average of the four. This scheme is therefore the same as bilinear interpolation. The matrix in eq. (164) is also referred to as the *symbol* of $\mathcal{P}$.

For the reverse operation, Restriction, The target points already have pre-existing values. One possible choice is to simply use those values and ignore the values at points being discarded. This is called direct injection and the corresponding *symbol* will be [1]. However, a better choice that uses more of the information from the finer level is $\mathcal{R}$ with the symbol

$$\begin{bmatrix} \frac{1}{16} & \frac{1}{8} & \frac{1}{16} \\ \frac{1}{8} & \frac{1}{4} & \frac{1}{8} \\ \frac{1}{16} & \frac{1}{8} & \frac{1}{16} \end{bmatrix} \quad (165)$$

which is an adjoint of $\mathcal{P}$. This is called the *full weight* version of $\mathcal{R}$. Sometimes a *half-weight* version, which is mid-way between full weight and direct

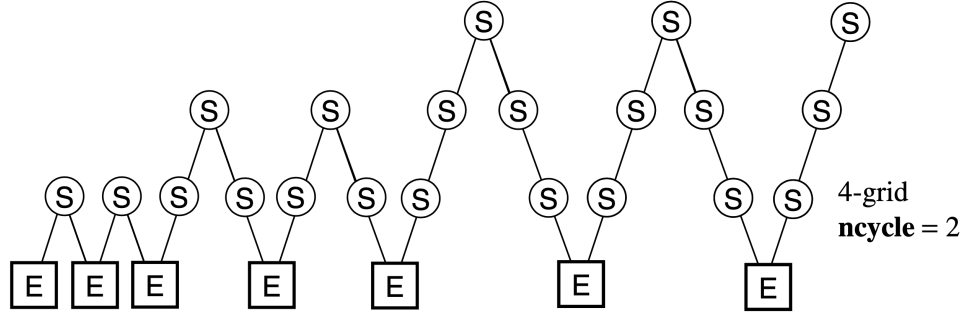


Figure 2: Illustration of the Full Multi-Grid (FMG) method. The solution starts on the coarsest grid and gradually proceeds to finer grids. Operation proceeds from the left to the right in this diagram. **ncycle** refers to the number of cycles traversed at each level before proceeding to a finer level. Figure taken from *Numerical Recipes*, 3rd Ed, by W.H. Press *et al*, 2007, Cambridge University Press.

injection, is also used. The corresponding symbol is:

$$\begin{bmatrix} 0 & \frac{1}{8} & 0 \\ \frac{1}{8} & \frac{1}{2} & \frac{1}{8} \\ 0 & \frac{1}{8} & 0 \end{bmatrix} \quad (166)$$

Another variant of this method, called the *Full Multi-Grid* (FMG) method, starts by defining the problem at the coarsest level. An exact solution is obtained at the first step. The solution is then prolonged to one level finer scale, smoothed and coarse grained again. Once a desired accuracy is achieved at this level, the solution is prolonged to the next finer level and so on. This gradually moves to finer and finer scale through taller and taller structures as illustrated in Fig. 2. In the FMG method, f is usually discretized afresh at every new mesh level. The solution is produced at all the levels, so it is possible to use Richardson extrapolation in conjunction with FMG.

Spectral Methods for PDEs:

We will now discuss Spectral Methods for solving Partial Differential Equations. In finite difference methods, a differential equation of the form

$$\mathcal{L}u = f \quad (167)$$

was approximated by approximating the operator $\mathcal{L}$, defining the operations on a grid. In spectral methods, the solution u is instead approximated, typically by expanding it in an orthogonal basis and then keeping only a few terms in the expansion.

Let $\phi_n(x)$ represent the orthogonal basis of choice. If $x = [a, b]$ is the domain of definition of the basis, then a scalar product of the form

$$\langle \phi_m | \phi_n \rangle \equiv \int_a^b \phi_m^*(x) \phi_n(x) W(x) dx \quad (168)$$

must exist, where $W(x)$ is a weight function specific to the chosen basis. Orthogonality demands

$$\langle \phi_m | \phi_n \rangle = A_{(m)(n)} \delta_{mn} \quad (169)$$

and if the basis is orthonormal then

$$\langle \phi_m | \phi_n \rangle = \delta_{mn} \quad (170)$$

The essence of the spectral method is to expand the desired solution $u(x)$ in the chosen orthogonal basis

$$u(x) = \sum_{n=0}^{\infty} c_n \phi_n(x) \quad (171)$$

and approximate it by keeping only the first N terms

$$u_N(x) = \sum_{n=0}^N c_n \phi_n(x) \approx u(x) \quad (172)$$

such that the rms error $[\langle u|u \rangle - \langle u_N|u_N \rangle]^{1/2}$ is below a specified threshold. Here the angular brackets denote scalar products defined with the same weight function and boundaries as for the basis functions $\phi_n(x)$.

Popular choices of orthogonal bases include:

1. Trigonometric Functions:

$$\phi_n(x) = e^{inx} \quad (173)$$

Here $[a, b] = [0, 2\pi]$ and $W(x) = 1$. Periodic boundary conditions apply at the ends of the domain. The scalar product

$$\langle \phi_m | \phi_n \rangle = \int_0^{2\pi} \phi_m^*(x) \phi_n(x) dx = 2\pi \delta_{mn} \quad (174)$$

Expansion in this basis gives a Fourier Series.

All the functions mentioned below share the property that the basis function ϕ_n is a polynomial of degree n .

2. Legendre Functions:

$$P_n(x) = \frac{1}{2^n n!} \frac{d^n}{dx^n} (x^2 - 1)^n \quad (175)$$

Here $[a, b] = [-1, 1]$ and $W(x) = 1$. The scalar product

$$\langle P_m | P_n \rangle = \int_{-1}^1 P_m(x) P_n(x) dx = \frac{2}{n+1} \delta_{mn} \quad (176)$$

Recurrence relation:

$$(n+1)P_{n+1}(x) = (2n+1)xP_n(x) - nP_{n-1}(x) \quad (177)$$

The first few polynomials are

$$1, x, \frac{1}{2}(3x^2 - 1), \dots \quad (178)$$

Derivative relation:

$$(2n+1)P_n(x) = P'_{n+1}(x) - P'_{n-1}(x) \quad (179)$$

3. Chebyshev Functions:

$$T_n(x) = \cos[n \cos^{-1} x] \quad (180)$$

Here $[a, b] = [-1, 1]$ and $W(x) = 1/(\sqrt{1-x^2})$. The scalar product

$$\langle T_m | T_n \rangle = \int_{-1}^1 T_m(x) T_n(x) \frac{dx}{\sqrt{1-x^2}} = \frac{\pi}{(2-\delta_{n0})} \delta_{mn} \quad (181)$$

Note that Chebyshev polynomials are essentially trigonometric functions

$$T_n(\cos \theta) = \cos(n\theta),$$

so the expansion coefficients can be efficiently computed using Fast Fourier Transform.

Recurrence relation:

$$T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x) \quad (182)$$

with

$$T_0(x) = 1, \quad T_1(x) = x \quad (183)$$

Derivative relation:

$$2T_n(x) = \frac{T'_{n+1}(x)}{n+1} - \frac{T'_{n-1}(x)}{n-1} \quad (184)$$

4. Laguerre Functions:

$$L_n(x) \text{ with } [a, b] = [0, \infty] \text{ and } W(x) = e^{-x} \quad (185)$$

Recurrence relation

$$(n+1)L_{n+1}(x) = (2n+1-x)L_n(x) - nL_{n-1}(x) \quad (186)$$

with the first few polynomials

$$1, (-x+1), \frac{1}{2}(x^2-4x+2), \dots \quad (187)$$

Derivative relation:

$$xL'_n(x) = nL_n(x) - nL_{n-1}(x) \quad (188)$$

Scalar product:

$$\langle L_m | L_n \rangle = \int_0^\infty L_m(x) L_n(x) e^{-x} dx = \delta_{mn} \quad (189)$$

These polynomials are orthonormal.

5. Hermite Functions

$$H_n(x) \text{ with } [a, b] = [-\infty, \infty] \text{ and } W(x) = e^{-x^2} \quad (190)$$

Recurrence relation

$$H_{n+1}(x) = 2xH_n(x) - 2nH_{n-1}(x) \quad (191)$$

with the first few polynomials

$$1, 2x, 4x^2 - 2, \dots \quad (192)$$

Derivate relation:

$$H'_n(x) = 2nH_{n-1}(x) \quad (193)$$

Scalar product:

$$\langle H_m | H_n \rangle = \int_{-\infty}^{\infty} H_m(x) H_n(x) e^{-x^2} dx = 2^n n! \sqrt{\pi} \delta_{mn} \quad (194)$$

The task of solving the differential equation then translates to evaluating the appropriate approximating coefficients c_n in eq. (172). In fact the coefficients of expansion of $u_N(x)$ need not exactly be the same as the corresponding c_n -s in the infinite expansion of $u(x)$ in eq. (171), as a slight readjustment of coefficients in the truncated expansion may serve to reduce the residual error.

The success of a spectral method depends on finding a suitable expansion where convergence is achieved rapidly for a given problem, thus limiting the required number of terms in the expansion. With appropriate choice of basis functions and N , the error can be made to decrease exponentially with N .

Example

Consider the 1-d wave equation

$$\frac{\partial u}{\partial t} = \frac{\partial u}{\partial x} \quad (195)$$

with periodic boundary condition on $x : [0, 2\pi]$ and an initial condition

$$u(t = 0, x) = f(x) \quad (196)$$

Expanding in Fourier series,

$$u(t, x) = \sum_{n=-\infty}^{\infty} c_n(t) e^{inx} \quad (197)$$

which, upon application of the PDE yields

$$\frac{dc_n}{dt} = inc_n \quad (198)$$

So

$$c_n(t) = c_n(0) e^{int} \quad (199)$$

While $c_n(0)$ are obtained from the initial condition. Expanding

$$f(x) = \sum_{n=-\infty}^{\infty} f_n e^{inx} \quad (200)$$

one sees that

$$c_n(0) = f_n \quad (201)$$

As an example, if

$$f(x) = \sin(\pi \cos x) \quad (202)$$

then the analytical solution of the PDE is

$$u(t, x) = \sin[\pi \cos(x + t)] \quad (203)$$

In this case the spectral coefficients are

$$c_n(0) = (-1)^n J_n(\pi), \quad n \text{ odd} \quad (204)$$

where the Bessel functions $J_n(\pi)$ go exponentially to zero with n for $n > \pi$. So only a few terms in the spectral expansion will be able to represent the solution with high accuracy.

A choice of basis functions appropriate to the problem allows the convergence to be rapid. Typically, the following choices are popular:

- Fourier series for Periodic solutions
- For non-periodic solutions in rectangular bounded domains Chebyshev polynomials $T_n(x)$ are the basis of choice.

- For spherical geometry (r, θ, ϕ) , one may use $R(r)P_l(\cos \theta)e^{im\phi}$, where P_l -s are Legendre polynomials. The radial function $R(r)$ is often expanded on Chebyshev polynomials if the domain is bounded and not inclusive of $r = 0$. If $r = 0$ is included then more complex radial polynomials can be constructed. If the domain is infinite then Laguerre polynomials could be a possible choice.

Once the basis is chosen, there are multiple ways to then evaluate the expansion coefficients. One particularly efficient method is the pseudo-spectral method which bears resemblance to Gaussian Quadrature.

Gaussian Quadrature:

Gaussian Quadrature is a numerical approximation of an integral $\int_a^b g(x)dx$

$$\int_a^b g(x)dx = \int_a^b W(x)f(x)dx \approx \sum_{i=0}^N w_i f(x_i) \quad (205)$$

First, the function $g(x)$ is written as a product of two functions: a *weight function* $W(x)$ and a function $f(x)$ which is very well approximated by a polynomial. Second, one finds a set of $0 \dots N$ suitable abscissa x_i and weights w_i (total $2N + 2$ adjustable parameters) so that the approximate sum is *exact* for polynomials up to degree $2N + 1$. Note that $f(x_i)$ are f evaluated at the chosen abscissa x_i but w_i are additional parameters, *not* the weight function $W(x)$ evaluated at x_i . The evaluation of the original integral will therefore be

$$\int_a^b g(x)dx \approx \sum_{i=0}^N \frac{w_i}{W(x_i)} g(x_i) \quad (206)$$

The choice of the abscissas can be made as follows.

Define a set of orthogonal polynomials $p_j(x)$ such that for each j , there is only one polynomial of order j , and all polynomials in the set are mutually orthogonal over the specified weight function $W(x)$ and interval $[a, b]$:

$$\langle p_j | p_k \rangle = \int_a^b p_j(x)p_k(x)W(x)dx = \langle p_j | p_j \rangle \delta_{jk} \quad (207)$$

Consecutive polynomials may be constructed from a recurrence relation of the form

$$p_{-1}(x) = 0$$

$$\begin{aligned}
p_0(x) &= 1 \\
p_{j+1}(x) &= (x - a_j)p_j(x) - b_j p_{j-1}(x), \quad j = 0, 1, 2, \dots
\end{aligned} \tag{208}$$

where

$$a_j = \frac{\langle x p_j | p_j \rangle}{\langle p_j | p_j \rangle}, \quad j = 0, 1, 2, \dots \tag{209}$$

and

$$b_j = \frac{\langle p_j | p_j \rangle}{\langle p_{j-1} | p_{j-1} \rangle}, \quad j = 1, 2, \dots \tag{210}$$

The coefficient b_0 is arbitrary and may be set to zero.

The polynomial $p_j(x)$ may be shown to have exactly j distinct roots in the interval $[a, b]$, and these roots interleave those of p_{j-1} .

The abscissas x_i of the $N + 1$ -point Gaussian quadrature defined in eq. (205) are the zeros of the polynomial $p_{N+1}(x)$ of the above sequence.

The corresponding weights are

$$w_i = \frac{\langle p_N | p_N \rangle}{p_N(x_i) p'_{N+1}(x_i)} \tag{211}$$

Depending on the domain and the boundary conditions, orthogonal polynomials such as Legendre, Chebyshev, Laguerre, Hermite etc may be used for Gaussian quadrature. For Chebyshev polynomials, the abscissas and weights can be analytically expressed:

$$x_i = \cos\left(\frac{\pi(i + \frac{1}{2})}{N + 1}\right), \quad w_i = \frac{\pi}{N + 1} \tag{212}$$

with $W(x) = (1 - x^2)^{-1/2}$ and the domain $-1 < x < 1$. In other cases the abscissa and the weights may be evaluated numerically.

Implementation of **Collocation or Pseudo-Spectral Method**:

1. Choose an appropriate orthonormal basis $\phi_i(x)$ depending on the problem geometry, boundary conditions etc.
2. For an equation of the type

$$\mathcal{L}y(x) = f(x) \quad (213)$$

expand

$$y(x) = \sum_{n=0}^{\infty} c_n \phi_n(x) \quad (214)$$

and define the *interpolating* polynomial

$$P_N(x) = \sum_{n=0}^N \bar{c}_n \phi_n(x) \quad (215)$$

such that at $N + 1$ collocation point x_j ,

$$P_N(x_j) = y(x_j), \quad j = 0, 1, \dots, N \quad (216)$$

i.e.

$$\sum_{n=0}^N \bar{c}_n \mathcal{L}\phi_n(x_j) = f(x_j) \quad (217)$$

3. The above is a set of linear equations of the form

$$[L] \cdot \vec{c} = \vec{f} \quad (218)$$

where

$$L_{jn} = \mathcal{L}\phi_n(x_j) \quad (219)$$

which may be solved via matrix methods to yield $\bar{c}_n$, which is the desired solution. In carrying this out, boundary conditions will need to be imposed as additional constraints on $\bar{c}_n$.

The common choice of collocation points are the Gaussian Quadrature points, namely the zeros of ϕ_{N+1} . In this case the coefficients $\bar{c}_n$ are given exactly by the discrete Gaussian scalar product

$$\bar{c}_n = \langle y | \phi_n \rangle_G \equiv \sum_{i=0}^N w_i y(x_i) \phi_n(x_i) \quad (220)$$

An equivalent approach can be adopted also in *Physical* space.

One may define a set of *Cardinal functions* $C_i(x)$ which are all polynomials of degree N , with the property that at the $N + 1$ collocation points

$$C_i(x_j) = \delta_{ij} \quad (221)$$

Possible ways to construct Cardinal functions include

a. Lagrange interpolators:

$$C_i(x) = \prod_{j=0, j \neq i}^N \frac{x - x_j}{x_i - x_j} \quad (222)$$

b. Removing the i -th zero from $\phi_{N+1}(x)$:

$$C_i(x) = \frac{\phi_{N+1}(x)}{(x - x_i)\phi'_{N+1}(x_i)}, \quad i = 0, 1, 2, \dots, N \quad (223)$$

Now the solution can be expressed as

$$y(x) = \sum_{i=0}^N y_i C_i(x) \quad (224)$$

then from the PDE,

$$\sum_{i=0}^N y_i \mathcal{L}C_i(x) = f(x) \quad (225)$$

so at collocation points x_j

$$\sum_{i=0}^N y_i \mathcal{L}C_i(x_j) = f(x_j) \quad (226)$$

which is a set of linear equations that can be solved to yield the coefficients y_i by taking into account the boundary conditions.

Often the boundary conditions are specified at the ends of the domain, so it is useful if the limits a, b may be included among the collocation points. This can be done by slight readjustment of the recurrence coefficients $a_N, a_{N-1}, b_N, b_{N-1}$ in eq. 208. The resulting collocation points are known as

Gauss-Lobatto points. This would allow directly substituting the boundary values at the ends of the domain as the corresponding values of y_i .

One may note that this method of solving the equation in physical space is akin to finite differencing. The difference here is that the mesh points are not equispaced, but rather chosen optimally to maximise the accuracy of the solution for a given number of grid points.

Galerkin Method:

If the solution $u(x)$ of a PDE $\mathcal{L}u(x) = f(x)$ is represented in terms of basis functions $\phi_n(x)$ as

$$u(x) = \sum_{n=0}^N c_n \phi_n(x) \quad (227)$$

then the Galerkin method provides a way to evaluate the coefficients c_n in the following manner.

Define the scalar product of functions $f(x)$ and $g(x)$ as

$$\langle f|g \rangle = \int_a^b f(x)g(x)dx \quad (228)$$

where $[a, b]$ is the domain of validity of the solution.

Let

$$\xi(x) = \mathcal{L}u(x) - f(x) \quad (229)$$

be the residual.

One then demands

$$\langle \phi_n|\xi \rangle = 0 \quad \text{for } n = 0, 1, 2, \dots, N \quad (230)$$

From the expansion, we see that ξ is composed of basis functions ϕ_n and their derivatives

$$\langle \phi_n|\xi \rangle = \sum_{m=0}^N c_m \langle \phi_n|\mathcal{L}\phi_m \rangle - \langle \phi_n|f \rangle = 0 \quad (231)$$

This is a set of linear equations which may be solved to evaluate c_m -s. This is known as the Galerkin method, which makes the residual orthogonal to as many basis functions as desired.

This technique can be applied to the Spectral method, and also other methods that expand the solution in a chosen basis. Among these is the Finite Element Method.

Finite Element Method:

An FEM solution proceeds thus:

- Divide the solution volume V into multiple elements as per convenience, each element could be of a different size.
- Approximate the solution within each element, typically by expressing in terms of piecewise polynomial basis functions, often linear functions.
- Each element may have multiple nodal points, and Lagrange interpolators may be used as the polynomial basis functions
- If there are N_E elements and each has n nodes, then the total number of nodes would be $nN_E + 1$, accounting for both boundaries. The solution is then expanded in $nN_E + 1$ basis functions

$$u \approx u_N = \sum_{m=0}^N u_m C_m \quad (232)$$

where $N = nN_E$ and the basis functions C_m are similar to Cardinal functions, each being unity at one nodal point and zero at others. The coefficients u_m , which determine the solution, can be evaluated via the Galerkin method.

The Finite Element method is one of the most popular and widely used methods for solving a variety of PDEs. One of the advantages of this method is that complex geometries can be handled easily. Different material properties in different elements can be incorporated.

Simple examples of Finite Element Method:

Consider an equation

$$\frac{d^2 u(x)}{dx^2} = f(x) \quad (233)$$

in the domain $x = [0, 1]$, with Boundary Conditions $u(0) = u(1) = 0$. We denote this as $\mathcal{L}u = f$.

We attempt to solve this by a finite element method.

First we decompose the domain $[0, 1]$ into discrete elements according to our convenience. The element sizes need not be equal (see fig. 3).

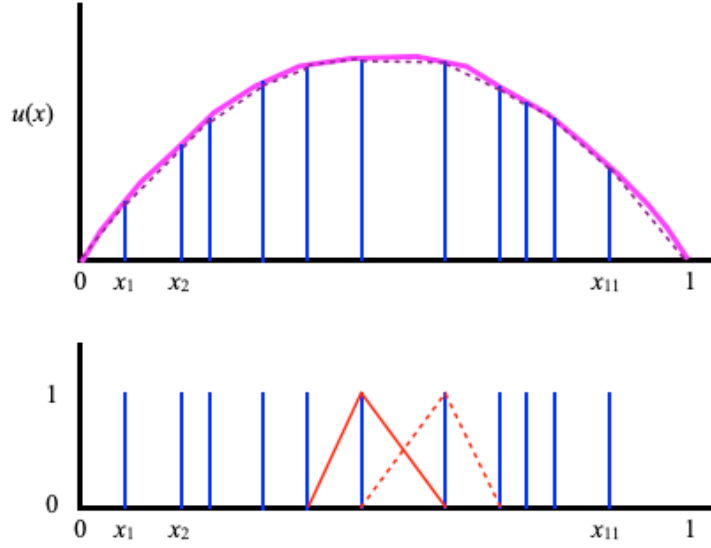


Figure 3: Finite element representation of the solution to eq. (233). The horizontal axis represents the independent variable x . The solution domain is $x = [0, 1]$ and the solution goes to zero at both ends of the domain. Upper panel: Element definitions and linear approximation (dashed lines) within each element of the solution (magenta). The vertical blue lines represent nodes $x_0 \cdots x_N$ (here $N = 12$). Lower panel: Interpolating polynomials (basis functions). Each function contains two linear segments, is unity at one of the nodes and is zero at all other nodes. Two such basis functions are shown in red.

Each boundary between adjacent elements represents a node. We choose to model the behaviour of the solution within each element as linear. In this case no additional internal nodes will be required. For expansion basis, we construct Cardinal functions with linear segments as follows:

$$C_k(x) = \begin{cases} \frac{x-x_k}{x_k-x_{k-1}} & x_{k-1} \leq x < x_k \\ \frac{x_{k+1}-x}{x_{k+1}-x_k} & x_k \leq x \leq x_{k+1} \\ 0 & \text{otherwise} \end{cases} \quad (234)$$

Examples of these interpolating polynomials, also called *Tent Functions*, are shown in the lower panel of fig. 3.

We note that the above Cardinal functions obey the specified boundary conditions, that their scalar product

$$\langle C_j | C_k \rangle \equiv \int_0^1 C_j(x) C_k(x) dx \approx \delta_{jk} \langle C_k | C_k \rangle \quad (235)$$

and also that

$$\int_0^1 C'_j(x) C'_k(x) dx \approx 0 \text{ for } j \neq k. \quad (236)$$

Note that both the quantities above vanish for $|j - k| > 1$.

We now expand the desired solution in this basis:

$$u(x) = \sum_{k=0}^N u_k C_k(x) \quad (237)$$

and as in Galerkin Method, demand that

$$\int_0^1 C_j(x) [\mathcal{L}u - f] dx = 0, \quad j = 0 \cdots n. \quad (238)$$

Integrating by parts,

$$C_j(x) u'(x) \Big|_0^1 - \int_0^1 [C'_j(x) u'(x) + C_j(x) f(x)] dx = 0. \quad (239)$$

The first term vanishes as the C_j -s satisfy the boundary conditions. Thus

$$\int_0^1 C'_j(x) u'(x) dx = - \int_0^1 C_j(x) f(x) dx \equiv b_j. \quad (240)$$

The RHS can be evaluated to yield the vector $\vec{b}$. The LHS may be written as

$$\sum_{k=0}^N u_k \int_0^1 C'_j(x) C'_k(x) dx \equiv \sum_{k=0}^n S_{jk} u_k \quad (241)$$

yielding a matrix equation

$$[S] \cdot \vec{u} = \vec{b} \quad (242)$$

which can be solved to obtain the expansion coefficients u_k . In this specific example $[S]$ is tridiagonal.

An FEM example: Assembly of springs

We now illustrate the basic algorithmic steps involved in Finite Element Analysis, via a simple one-dimensional example of an assembly of springs, in which each spring may be considered an element. The physics problem is to connect the displacements and forces, and in this problem the relationship between them is linear. So a linear description within an element is exact.

For a single spring as in figure 4, the system may be described using two nodes 1 and 2, at which the displacements are x_1 and x_2 and loads (forces) are R_1 and R_2 respectively.

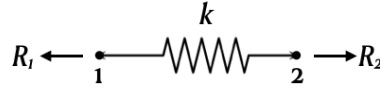


Figure 4: An element comprised of a single spring of stiffness k

Given the spring stiffness k , and noting that in equilibrium $R_1 + R_2 = 0$, we have the relations

$$\begin{aligned} R_1 &= kx_1 - kx_2 \\ R_2 &= kx_2 - kx_1 \end{aligned}$$

We may write this as

$$\begin{bmatrix} k & -k \\ -k & k \end{bmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} R_1 \\ R_2 \end{pmatrix}$$

or

$$[K] \cdot \vec{x} = \vec{R} \quad (243)$$

where $[K]$ is called the stiffness matrix, $\vec{x}$ the displacement vector and $\vec{R}$ the load vector. The above relation can be specified for each element of the spring assembly, with respective stiffness matrices.

Let us now consider a full assembly, as shown in fig. 5. We can decompose this into four individual elements, with each described by a local matrix equation of the form eq. (243). The local stiffness matrix

$$[K]^{(e)} = \begin{bmatrix} k_{11}^{(e)} & k_{12}^{(e)} \\ k_{21}^{(e)} & k_{22}^{(e)} \end{bmatrix} \quad (244)$$

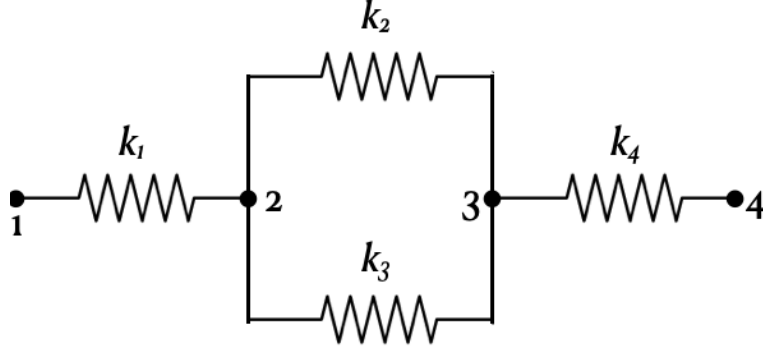


Figure 5: An assemblage of springs for a one-dimensional finite element analysis.

is specific to the corresponding element properties.

We decompose the arrangement in fig. 5 into four elements with spring stiffness k_1, k_2, k_3 and k_4 respectively. We then have

$$\begin{aligned}
 [K]^{(1)} : k_{11}^{(1)} &= -k_{12}^{(1)} = -k_{21}^{(1)} = k_{22}^{(1)} = k_1 \\
 [K]^{(2)} : k_{11}^{(2)} &= -k_{12}^{(2)} = -k_{21}^{(2)} = k_{22}^{(2)} = k_2 \\
 [K]^{(3)} : k_{11}^{(3)} &= -k_{12}^{(3)} = -k_{21}^{(3)} = k_{22}^{(3)} = k_3 \\
 [K]^{(4)} : k_{11}^{(4)} &= -k_{12}^{(4)} = -k_{21}^{(4)} = k_{22}^{(4)} = k_4
 \end{aligned}$$

Each element has its own nodes 1 and 2 defined. The next task is to define the system topology, which allows one to embed the local coordinates of each node into the global arrangement. For this, we locate the nodes of the full system and assign them global identification numbers. In this problem, we need four nodes, numbered 1,2,3,4 as in fig. 5. The topology is defined by mapping the local nodes to the global IDs as in the following table:

The global displacement and load vectors would be

$$\vec{x}_g = (x_1 \ x_2 \ x_3 \ x_4)^T; \quad \vec{R}_g = (R_1 \ R_2 \ R_3 \ R_4)^T$$

The global stiffness matrix connecting them will then be compiled from the stiffness matrices of individual nodes by an *Assembly* process. This proceeds as follows. First, the local stiffness matrix elements are assigned

Element No.	Local nodeID	Global nodeID
1	1	1
	2	2
2	1	2
	2	3
3	1	2
	2	3
4	1	3
	2	4

global indices according to the topology. In the present case the following mapping occurs:

$$[K]^{(1)} = \begin{bmatrix} k_{11}^{(1)} & k_{12}^{(1)} \\ k_{21}^{(1)} & k_{22}^{(1)} \end{bmatrix}, \quad [K]^{(2)} = \begin{bmatrix} k_{22}^{(2)} & k_{23}^{(2)} \\ k_{32}^{(2)} & k_{33}^{(2)} \end{bmatrix}$$

$$[K]^{(3)} = \begin{bmatrix} k_{22}^{(3)} & k_{23}^{(3)} \\ k_{32}^{(3)} & k_{33}^{(3)} \end{bmatrix}, \quad [K]^{(4)} = \begin{bmatrix} k_{33}^{(4)} & k_{34}^{(4)} \\ k_{43}^{(4)} & k_{44}^{(4)} \end{bmatrix}$$

Each of these may then be situated into the global stiffness matrix structure:

$$[K]_g^{(1)} = \begin{bmatrix} k_{11}^{(1)} & k_{12}^{(1)} & 0 & 0 \\ k_{21}^{(1)} & k_{22}^{(1)} & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad [K]_g^{(2)} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & k_{22}^{(2)} & k_{23}^{(2)} & 0 \\ 0 & k_{32}^{(2)} & k_{33}^{(2)} & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$[K]_g^{(3)} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & k_{22}^{(3)} & k_{23}^{(3)} & 0 \\ 0 & k_{32}^{(3)} & k_{33}^{(3)} & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad [K]_g^{(4)} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & k_{33}^{(4)} & k_{34}^{(4)} \\ 0 & 0 & k_{43}^{(4)} & k_{44}^{(4)} \end{bmatrix}$$

The assembly process is then completed by summing these partial matrices:

$$[K]_g = \sum_e [K]_g^{(e)} \quad (245)$$

So we have, in the present case,

$$[K]_g = \begin{bmatrix} k_{11}^{(1)} & k_{12}^{(1)} & 0 & 0 \\ k_{21}^{(1)} & k_{22}^{(1)} + k_{22}^{(2)} + k_{22}^{(3)} & k_{23}^{(2)} + k_{23}^{(3)} & 0 \\ 0 & k_{32}^{(2)} + k_{32}^{(3)} & k_{33}^{(2)} + k_{33}^{(3)} + k_{33}^{(4)} & k_{34}^{(4)} \\ 0 & 0 & k_{43}^{(4)} & k_{44}^{(4)} \end{bmatrix}$$

The elements of which we will denote as $k_{ij}^{(g)}$. One is now ready to solve for the displacements $\vec{x}_g$, given the external loads $\vec{R}_g$.

However, it can be easily seen that the matrix $[K]_g$ has zero determinant, is thus singular and cannot be inverted. To overcome this, one needs to apply boundary conditions.

Let the boundary conditions be specified as two of the displacements, for example $x_1 = \beta_1$ and $x_3 = \beta_3$. The algorithm used to incorporate them is as follows:

- If x_i at node i is specified to be β_i , then in the stiffness matrix $[K]_g$ set the non-diagonal elements of row i and column i to zero.
- Set the diagonal element $k_{ii}^{(g)}$ to unity.
- Replace R_i in the load vector by β_i .
- Adjust the other elements of the load vector to gather all the known quantities in the RHS.
- Repeat the above steps for all other specified x_i , one at a time.

Following the above prescription, the matrix equation of the present problem is transformed to:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & k_{22}^{(g)} & 0 & k_{24}^{(g)} \\ 0 & 0 & 1 & 0 \\ 0 & k_{42}^{(g)} & 0 & k_{44}^{(g)} \end{bmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} \beta_1 \\ R_2 - k_{21}^{(g)}\beta_1 - k_{23}^{(g)}\beta_3 \\ \beta_3 \\ R_4 - k_{41}^{(g)}\beta_1 - k_{43}^{(g)}\beta_3 \end{pmatrix} \quad (246)$$

The modified stiffness matrix is now invertible and can be used to obtain the final solution.

Recommended reading:

The Finite Element Method for Engineers – K.H. Huebner, D.L. Dewhirst,
D.E. Smith and T.G. Byron, Wiley

Finite Volume Method:

The Finite Volume Method has seen widespread use in HD and MHD problems in Astrophysics, because of its flexibility to model complex geometries, handle discontinuities like shock waves and for naturally implementing conservation laws.

The method discretizes the computational domain into volume elements called *cells*, which need not be of equal size.

For example, in a 1-d problem one may imagine an unit area transverse to the problem dimension to be attached, and volumes defined accordingly. If the independent variable is x , then the discretization may be done by splitting the problem into cells of different widths in x . A cell i would have central coordinate x_i and boundaries at $x_{i-1/2}$ and $x_{i+1/2}$

The quantities solved for in this method are the volume averages of the dependent variables in each cell, in terms of the fluxes of the quantity at the cell boundaries.

The nature of problems this is suited to is the flux conservative initial value problem of the type

$$\frac{\partial u}{\partial t} + \vec{\nabla} \cdot \vec{F} = 0 \quad (247)$$

The fluid equations are therefore cast in their conservative form:

$$\frac{\partial \rho}{\partial t} + \vec{\nabla} \cdot (\rho \vec{v}) = 0 \quad [\text{mass}] \quad (248)$$

$$\frac{\partial(\rho \vec{v})}{\partial t} + \vec{\nabla} \cdot (\rho [v_i v_j] + P[\delta_{ij}]) = 0 \quad [\text{momentum}] \quad (249)$$

$$\frac{\partial \xi}{\partial t} + \vec{\nabla} \cdot [(\xi + P)\vec{v}] = 0 \quad [\text{energy}] \quad (250)$$

where

$$\xi = \rho e + \frac{1}{2} \rho v^2 = \rho e + \frac{(\rho \vec{v})^2}{2\rho} \quad (251)$$

e being the internal energy per unit mass.

The variables to be solved for are then ρ , $\rho \vec{v}$ and ξ , noting that the above equations would be supplemented by an equation of state of the form $e = f(P, \rho)$.

Considering a problem in 1-d, the average of the dependent variable in a

cell i would be defined as

$$\langle u \rangle_i(t) = \frac{1}{x_{i+1/2} - x_{i-1/2}} \int_{x_{i-1/2}}^{x_{i+1/2}} u(x, t) dx \quad (252)$$

where in the limits $x_{i-1/2}$ and $x_{i+1/2}$ have been abbreviated to $i - 1/2$ and $i + 1/2$ respectively.

For a time step n ,

$$\langle u \rangle_i^n = \frac{1}{\Delta x} \int_{x_{i-1/2}}^{x_{i+1/2}} u(x, t^n) dx \quad (253)$$

where $\Delta x = x_{i+1/2} - x_{i-1/2}$.

Now as

$$\frac{\partial u}{\partial t} + \frac{\partial F}{\partial x} = 0 \quad (254)$$

one may write the integral

$$\int_{t^n}^{t^{n+1}} \int_{x_{i-1/2}}^{x_{i+1/2}} \left(\frac{\partial u}{\partial t} + \frac{\partial F}{\partial x} \right) dx dt = 0 \quad (255)$$

So

$$\int_{t^n}^{t^{n+1}} \left[\Delta x \frac{d\langle u \rangle_i}{dt} + (F_{i+1/2} - F_{i-1/2}) \right] dt = 0 \quad (256)$$

Hence

$$\langle u \rangle_i^{n+1} = \langle u \rangle_i^n - \frac{\Delta t}{\Delta x} [\tilde{F}_{i+1/2}^{n+1/2} - \tilde{F}_{i-1/2}^{n+1/2}] \quad (257)$$

where $\tilde{F}$ -s are time averages. They are normally replaced by fluxes evaluated at half time steps which are considered to be constant through the time step.

This manner of linking the volume average to the fluxes at the cell boundaries is known as the *Godunov scheme*.

The time stepping can be performed by any method of choice such as Runge-Kutta.

It is to be noted that while the fluxes $\tilde{F}(u, t)$ are evaluated at the cell boundaries, and at half time steps, the variable $\langle u \rangle$ is known at cell centres and at integer time steps. Hence a suitable propagation scheme needs to be adopted.

To evaluate u at the cell boundary that the Flux is a function of, at zeroeth order one may just take the average of left and right $\langle u \rangle$ values at each

boundary. Higher order polynomial extrapolations within a cell could also be resorted to if desired.

Once the flux values at the cell boundaries are known at the time t^n , the task would then be to determine the values at $t^{n+1/2}$. For this, the solution of a *Riemann problem* is derived as follows.

Consider a purely advective equation

$$\frac{\partial u}{\partial t} = -a \frac{\partial u}{\partial x} \quad (258)$$

where $a=\text{constant}$ is the propagation speed. The solution of this is of the form

$$u = f(x - at) \quad (259)$$

i.e. the solution just shifts in time at the speed a , to the right if $a > 0$ and to the left if $a < 0$, along a *characteristic* of slope a .

Now our desired problem is that of a set of m coupled PDEs,

$$\frac{\partial \vec{u}}{\partial t} + \frac{\partial \vec{F}(\vec{u})}{\partial x} = 0 \quad (260)$$

where the vector components correspond to the different dependent variables $u_1 \cdots u_m$ that need to be solved for simultaneously.

We may rewrite the above as

$$\frac{\partial \vec{u}}{\partial t} + \frac{\partial \vec{F}}{\partial \vec{u}} \frac{\partial \vec{u}}{\partial x} = 0 \quad (261)$$

where $\partial \vec{F} / \partial \vec{u}$ is a Jacobian matrix

$$[A] = \begin{bmatrix} \partial F_1 / \partial u_1 & \cdots & \partial F_1 / \partial u_m \\ \vdots & & \vdots \\ \partial F_m / \partial u_1 & \cdots & \partial F_m / \partial u_m \end{bmatrix} \quad (262)$$

So

$$\frac{\partial \vec{u}}{\partial t} + [A] \cdot \frac{\partial \vec{u}}{\partial x} = 0 \quad (263)$$

Now say $\lambda_1 \cdots \lambda_m$ are the eigenvalues of $[A]$ (in ascending order of magnitude), and $\vec{K}_1 \cdots \vec{K}_m$ the corresponding eigenvectors. If we construct a matrix

$$[K] = [K_1 \cdots K_m] \quad (264)$$

Then $[\Lambda] = [K]^{-1}[A][K]$ is a diagonal matrix with the eigenvalues $\lambda_1 \cdots \lambda_m$ as the diagonal elements.

If we define new variables

$$\vec{w} = [K]^{-1} \cdot \vec{u} \quad (265)$$

Then eq. (263) may be rewritten, substituting $\vec{u} = [K] \cdot \vec{w}$, and operating $[K]^{-1}$ from the left:

$$\frac{\partial \vec{w}}{\partial t} + [\Lambda] \frac{\partial \vec{w}}{\partial x} = 0 \quad (266)$$

This uncouples the equations, and each new variable $w_1 \cdots w_m$ now propagates with a characteristic speed $\lambda_1 \cdots \lambda_m$, as in pure advection [eq. (258)]. Therefore

$$w_k(x, t) = w_k^0(x - \lambda_k t); \quad [w_k^0(x) \equiv w_k(x, t = 0)] \quad (267)$$

gives the entire solution field, from which the original variables can then be reconstructed:

$$\vec{u} = [K] \cdot \vec{w} \quad (268)$$

In Godunov scheme, the quantities of interest are the volume averages for each cell. At the boundary of cell i , let the initial values of the variable u_j on the two sides be

$$u_j(x, 0) = \begin{cases} u_{j,L} & \text{for } x < x_{i-1/2} \\ u_{j,R} & \text{for } x > x_{i-1/2} \end{cases} \quad (269)$$

$$j = 1 \dots m$$

In terms of the eigenvectors $\vec{K}_l$ of the Jacobian matrix, these can be written as

$$\vec{u}_L = \sum_{l=1}^m \alpha_l \vec{K}_l, \quad \vec{u}_R = \sum_{l=1}^m \beta_l \vec{K}_l \quad (270)$$

Then

$$w_l^0(x) = \begin{cases} \alpha_l & \text{for } x < 0 \\ \beta_l & \text{for } x > 0 \end{cases} \quad (271)$$

setting the x -origin of the local problem at the cell boundary.

So

$$w_l(x, t) = w_l^0(x - \lambda_l t) = \begin{cases} \alpha_l & \text{if } x - \lambda_l t < 0 \\ \beta_l & \text{if } x - \lambda_l t > 0 \end{cases} \quad (272)$$

Hence, for $\lambda_I < x/t < \lambda_{I+1}$,

$$\vec{u}(x, t) = \sum_{l=I+1}^m \alpha_l \vec{K}_l + \sum_{l=1}^I \beta_l \vec{K}_l \quad (273)$$

where the origin of time is t^n .

In a 1-d hydrodynamic problem with three state variables ρ , ρv and ξ , $m = 3$ and one will obtain three different characteristics with different propagation speeds.

The solution is often simplified by considering just the fastest and the slowest speeds and ignoring the middle wave [Hartex-Lax-van Leer (HLL) solver], Many other approaches and approximations have also been devised and are applied according to the demands of the problem.

The Riemann solution of eq. (273) is employed to evaluate $\vec{u}$ at cell boundaries at $t^{n+1/2}$, which then yield the fluxes $\tilde{F}_{i+1/2}^{n+1/2}$. These values are then used to propagate the solution to t^{n+1} using eq. (257).

One of the extensively used packages for Astrophysical fluid dynamics and MHD is PLUTO. The download link and some associated tutorial material may be accessed at the URL <https://web.iucaa.in/~dipankar/CMA2020/MD.html>

N-body simulation:

N-body simulations are used extensively in Astrophysics to model the dynamics of star clusters, galaxies and the dark matter in the universe, among others. This is applicable largely to situations where gravity makes the dominant contribution to the dynamics and fluid contributions such as pressure, viscosity etc need not be explicitly considered.

The method is conceptually simple: the mass distribution is modelled as a collection of discrete macroscopic particles, and their dynamics is followed by computing the mutual gravitational forces upon each other. Each particle represents a portion of the system to be simulated. A particle could represent a star, a galaxy or even a collection of stars or galaxies, as well as a finite quantity of dark matter. The number of macroscopic particles that can be included will be limited by the computing resources available for a simulation.

For each particle in the system

$$\frac{d^2 \vec{r}_i}{dt^2} = -\vec{\nabla}_i \phi \quad (274)$$

$$\vec{F}_i = -\vec{\nabla}_i \phi = - \sum_{j \neq i} \frac{G m_i (\vec{r}_i - \vec{r}_j)}{|\vec{r}_i - \vec{r}_j|^3} \quad (275)$$

where ϕ is the gravitational potential, m_i is the mass of the i -th particle, $\vec{r}_i$ its position and $\vec{F}_i$ is the gravitational force on it. $\vec{\nabla}_i$ represents the gradient evaluated at the position of the particle i .

In order to avoid the singularity at $\vec{r}_i = \vec{r}_j$, a softening of the potential is often introduced:

$$\phi(\vec{r}) = -G \sum_{i=1}^N \frac{m_i}{\sqrt{|\vec{r} - \vec{r}_i|^2 + \epsilon^2}} \quad (276)$$

Tree algorithm (Barnes and Hut 1986):

- Hierarchically subdivide the volume into nodes (*octants*) until the node contains either one or zero particle.
- Nodes with one or zero particle are called “external”. They are not further sub-divided.
- Nodes with more than one particle are “internal”. They are sub-divided into octants with origin at the centre of the node.

- At each stage of the hierarchy, the total mass, the position of the centre of mass and the coordinates of all member particles of each node are stored.
- For each particle, consider the angle θ subtended to it by each node.
- If $\theta < \theta_0$ (a threshold parameter), then compute the gravitational force due to the total mass of the node situated at its centre of mass. Further subdivisions of this node are ignored.
- If $\theta > \theta_0$ then gravitational force due to member elements of the node are individually computed.

This algorithm brings down the number of force evaluations from of order N^2 to of order $N \log_2 N$.

Particle Mesh (PM) algorithm:

- Create a Mesh
- Distribute the mass of each particle to the mesh points adjacent to its location.
- Sum the total mass contribution at each mesh point. Convert this to a density field ρ
- Solve Poisson's equation on the mesh:

$$\nabla^2 \phi = 4\pi G \rho \quad (277)$$

This is a boundary value problem on a mesh, and is often solved by the Fourier transform method.

- From the potential, evaluate the force at the location of each particle. If the Poisson equation has been solved in the Fourier domain to yield $\hat{\phi}(k)$, then the force in the spatial domain can be obtained by taking an inverse Fourier transform of $-ik\hat{\phi}(k)$.

In either algorithm, once the force on a particle is known,

- update velocity
- update position from velocity

and then repeat for the next time step.

Further methods have been devised to combine the Tree and PM algorithms into what is called a **TPM** (Xu 1995) or **TreePM** (Bagla 2002) algorithm. Here the Gravitational force is split into long-range and short-range contributions. The long-range potential is computed in Fourier space as in the PM algorithm, and the short-range force is computed directly in real space using the Tree algorithm. This leverages the computing speed of the PM algorithm for long-range forces and avoids the large errors the PM algorithm is known to have at short ranges.

One challenge in N-body simulations is to decide on an appropriate time step. Due to a large range in density, the dynamical time scale $\tau \sim 1/\sqrt{G\rho}$ varies greatly from region to region. Adjusting the time step to the smallest dynamical time scale would be very inefficient for less dense regions.

A hierarchy of time steps can be resorted to:

- Arrange particles in hierarchy of time steps in factors of two: $\Delta t_n = \Delta t_0/2^n$
- Advance particles in the same block of timesteps simultaneously.
- For every two advances of a certain block the next higher block is advanced once.

In cosmological N-body simulations, time steps inversely related to acceleration is often used, e.g. $\Delta t_i = \eta \sqrt{\frac{\epsilon}{|a_i|}}$, where η and ϵ are chosen constants. Higher order integrators may use time steps related to derivatives of the acceleration.

References:

1. Barnes, J. and Hut, P. (1986), *Nature*, **324**, 446
2. Xu, G. (1995), *Astrophysical Journal Supplement Series*, **98**, 355
3. Bagla, J.S. (2002), *Journal of Astrophysics and Astronomy*, **23**, 1

Example N-body codes:

Sverre Aarseth's code (Cambridge):

<https://people.ast.cam.ac.uk/~sverre/web/pages/nbody.htm>

GADGET code:

<https://www.h-its.org/2014/10/29/gadget-code>

<https://www.mpa-garching.mpg.de/galform/gadget/>

Smoothed Particle Hydrodynamics (SPH):

In Smoothed Particle Hydrodynamics, a fluid is modelled as a collection of discrete particles, each particle representing a portion of the fluid. Each particle is assigned its own attributes such as mass, volume, pressure, velocity, position etc. A smoothing kernel is then specified. The contribution of each particle across the full computational domain is implemented through this smoothing kernel. Quantities that govern the evolution at any given point are evaluated as averages over all particles weighted by the corresponding smoothing kernels. All spatial derivatives are implemented through the derivatives of the kernel functions. The width of the kernel function is user-adjustable and may vary in space and time.

Let us consider a fluid quantity $A(\vec{r})$. The SPH representation of it would be:

$$\begin{aligned} A(\vec{r}) &= \int A(\vec{r}') W(|\vec{r} - \vec{r}'|, h) dV \\ &= \sum_j A_j W(|\vec{r} - \vec{r}_j|, h_j) V_j \end{aligned} \quad (278)$$

where A_j is the quantity attributed to particle j , V_j its volume and $\vec{r}_j$ its position. W is the smoothing kernel.

As the dynamics is modelled by moving each particle individually, quantities that influence the dynamics are required to be evaluated at the location of each particle:

$$A(\vec{r}_i) = \sum_j A_j W(|\vec{r}_i - \vec{r}_j|, h_{ij}) V_j = \sum_j A_j W_{ij} V_j \quad (279)$$

Here $h_{ij} = (h_i + h_j)/2$.

One can then write

$$\rho_i = \sum_j m_j W_{ij} \quad (280)$$

$$P_i = \sigma_s^2 \sum_j m_j W_{ij} \quad (281)$$

where the fluid has been assumed to be isothermal, with σ_s as the speed of sound. m_j is the mass of the j th particle and ρ_i , P_i are the density and pressure of the fluid evaluated at the location of the i th particle.

Since each particle is assigned a non-varying mass and the motion of only these particles are tracked, conservation of mass is assured. The fluid equations are written as follows:

Continuity Equation

$$\frac{\partial \rho}{\partial t} + \vec{\nabla} \cdot (\rho \vec{v}) = 0$$

as

$$\frac{d\rho_i}{dt} = \sum_j m_j (\vec{v}_i - \vec{v}_j) \cdot \vec{\nabla}_i W_{ij} \quad (282)$$

$\vec{\nabla}_i W_{ij}$ is the gradient of W_{ij} with respect to the location of the i th particle. Note that $W_{ij} = W_{ji}$, and $\vec{\nabla}_i W_{ij} = -\vec{\nabla}_j W_{ji}$.

The Momentum Equation is

$$\frac{D\vec{v}}{Dt} = -\frac{\vec{\nabla} P}{\rho} - \vec{\nabla} \phi = -\left[\vec{\nabla} \left(\frac{P}{\rho} \right) + \frac{P}{\rho^2} \vec{\nabla} \rho \right] - \vec{\nabla} \phi$$

where the LHS is the convective derivative, as is appropriate for a moving fluid element. This may be written in SPH form as

$$\frac{d\vec{v}_i}{dt} = -\sum_j m_j \left(\frac{P_i}{\rho_i^2} + \frac{P_j}{\rho_j^2} \right) \vec{\nabla}_i W_{ij} - \vec{\nabla}_i \phi(\vec{r}_i) \quad (283)$$

wherein $\vec{\nabla} \phi$ is to be computed from the density field. The gradient of (P/ρ) yields a surface term upon volume integration, which can be absorbed into the boundary conditions. These ODEs may then be integrated using a method of choice, for example Runge-Kutta, or even Riemann solvers.

The Kernel function $W(|\vec{r} - \vec{r}'|, h)$ is normalised as $\int W dV = 1$. Among the possible forms are

- Gaussian:

$$W(r, h) = \frac{1}{(\pi h^2)^{3/2}} e^{-(r^2/h^2)}$$

- Spline:

$$W(r, h) = \frac{1}{\pi h^3} \begin{cases} 1 - \frac{3}{2}x^2 + \frac{3}{4}x^3 & [0 < x < 1] \\ \frac{1}{4}(2-x)^3 & [1 < x < 2] \\ 0 & \text{otherwise} \end{cases}$$

where $x = r/h$.

It may be necessary to introduce artificial viscosity to smooth out discontinuities or singularities that may arise during the evolution. The velocity equation may be modified as

$$\frac{d\vec{v}_i}{dt} = - \sum_j m_j \left(\frac{P_i}{\rho_i^2} + \frac{P_j}{\rho_j^2} + \Pi_{ij} \right) \vec{\nabla}_i W_{ij} - \vec{\nabla}_i \phi(\vec{r}_i) \quad (284)$$

where Π_{ij} is a viscosity tensor that may be written as

$$\Pi_{ij} = \begin{cases} \frac{1}{\rho_{ij}} \left(-\alpha \sigma_s \mu_{ij} + \beta \mu_{ij}^2 \right) & [\vec{v}_{ij} \cdot \vec{r}_{ij} \leq 0] \\ 0 & \text{otherwise} \end{cases} \quad (285)$$

with

$$\mu_{ij} = \frac{h_{ij}(\vec{v}_i - \vec{v}_j)(\vec{r}_i - \vec{r}_j)}{(\vec{r}_i - \vec{r}_j)^2 + \epsilon^2 h_{ij}^2} \quad (286)$$

where ϵ is a chosen small parameter.

The boundary conditions specified on an arbitrarily shaped enclosing surface may be handled in one of several ways, such as

- Integral Neglect: Region beyond the boundary is ignored while evaluating the integrals over kernel functions.
- Fluid Extension: Volume beyond the boundary are populated by *ghost* particles, implementing the imposed boundary conditions.
- Boundary Integral: The volume integral beyond the boundary is converted into a surface integral and the boundary conditions are thereby imposed.

SPH has seen a wide variety of applications in Astrophysics (gasdynamics), Oceanography (water flow), Volcanology (magma flow) as well as solid mechanics.

There are several publicly available SPH codes, including

GADGET: Here the baryonic fluid is evolved via SPH while dark matter is evolved through N-body simulation.

Fluids v3 (runs on Windows platform): https://github.com/erwincoumans/fluids_v3

GPUSPH (for linux platform and NVIDIA GPUs): <https://www.gpusph.org>

Bayesian Inference:

Bayesian statistics derives from the basic rules of conditional probability:

$$P(X|I) + P(\bar{X}|I) = 1 \quad (287)$$

$$P(X, Y|I) = P(X|Y, I)P(Y|I) \quad (288)$$

Where $P(A|B)$ represents the probability that A is true given B . I stands for the entirety of the background information required to contextualise an event. $\bar{X}$ means “ X is false”.

Since in the lhs of eq. (288) X and Y are interchangeable,

$$P(Y, X|I) = P(Y|X, I)P(X|I) \quad (289)$$

and thus

$$P(Y|X, I)P(X|I) = P(X|Y, I)P(Y|I) \quad (290)$$

giving us the Bayes’ theorem:

$$P(X|Y, I) = \frac{P(Y|X, I)P(X|I)}{P(Y|I)} \quad (291)$$

In the context of scientific inference, Y may represent data D and X a hypothesis H under test, i.e.

$$P(H|D, I) = \frac{P(D|H, I)P(H|I)}{P(D|I)} \quad (292)$$

Here $P(D|H, I)$ is called the *Likelihood*, $P(H|I)$ the *Prior*, $P(D|I)$ the *Evidence* and $P(H|D, I)$ the *Posterior*. Note that $P(D|I) = \sum_H P(D|H, I)P(H|I)$.

An important operation in Bayesian analysis is *Marginalisation*. This amounts to reducing the dependency subspace of a multivariate distribution by integrating over variables that are not of interest.

For example,

$$P(X|I) = \int_{-\infty}^{+\infty} p(X, Y|I) dY \quad (293)$$

is known as Marginalisation over Y . Here p is the probability density, namely $p dY$ is the probability for the value of Y being in the range Y to $Y + dY$.

Examples of application of Bayesian analysis:

1. Game Show (Monty Hall problem):

Here a contestant is asked to choose one of three closed doors. Behind one of the doors there is a car. The contestant wins the car if their choice matches the one with the car.

The game proceeds as follows. The contestant chooses door no. 1. The host then opens door no. 3 and shows that there is no car behind it. The contestant is then asked to choose between retaining their original choice of door no. 1 or changing it to door no. 2. The question is whether changing the choice would enhance the probability of the contestant winning the car.

The answer to this can be arrived at by a straightforward Bayesian analysis.

Consider three hypotheses:

- H_1 : Car is behind door 1
- H_2 : Car is behind door 2
- H_3 : Car is behind door 3

A priori all three are equally probable, so the priors

$$P(H_1|I) = P(H_2|I) = P(H_3|I) = \frac{1}{3} \quad (294)$$

The Data is: Door 3 was opened when Door 1 was chosen. We can compute the likelihood of this happening for each of the three hypotheses:

$$\begin{aligned} P(D|H_3, I) &= 0 \\ P(D|H_1, I) &= \frac{1}{2} \text{ (either door 2 or door 3 could have been opened)} \\ P(D|H_2, I) &= 1 \text{ (opening door 3 would have been the only choice)} \end{aligned}$$

Then

$$\begin{aligned} P(D|I) &= P(D|H_1, I)P(H_1|I) + P(D|H_2, I)P(H_2|I) + P(D|H_3, I)P(H_3|I) \\ &= \frac{1}{2} \cdot \frac{1}{3} + 1 \cdot \frac{1}{3} = \frac{1}{2} \end{aligned}$$

Thus

$$P(H_1|D, I) = \frac{P(D|H_1, I)P(H_1|I)}{P(D|I)} = \frac{\frac{1}{2} \cdot \frac{1}{3}}{\frac{1}{2}} = \frac{1}{3} \quad (295)$$

and

$$P(H_2|D, I) = \frac{P(D|H_2, I)P(H_2|I)}{P(D|I)} = \frac{1 \cdot \frac{1}{3}}{\frac{1}{2}} = \frac{2}{3} \quad (296)$$

Clearly, changing the choice to door no. 2 will improve the chances by a factor of two.

2. Medical Test:

A patient tests positive for a rare disease with an occurrence rate in of 1 in 10^4 in a population. The test conducted has a 2% false positive rate and 1% false negative rate. What is the probability that the patient indeed has the disease?

Here the hypothesis H is that the patient has the disease. The a priori probability of this is $P(H|I) = 10^{-4}$. The data D is that the patient tests positive. The accuracy of the medical test provides us

$$\begin{aligned} P(D|H, I) &= 1 - 1\% = 0.99 \\ P(D|\bar{H}, I) &= 2\% = 0.02 \end{aligned}$$

so

$$\begin{aligned} P(D|I) &= P(D|H, I)P(H|I) + P(D|\bar{H}, I)P(\bar{H}|I) \\ &= 0.99 \times 10^{-4} + 0.02 \times (1 - 10^{-4}) = 0.020097 \end{aligned}$$

and hence

$$\begin{aligned} P(H|D, I) &= \frac{P(D|H, I)P(H|I)}{P(D|I)} \\ &= \frac{0.99 \times 10^{-4}}{0.020097} = 0.00493 \\ &= 0.493\% \end{aligned} \quad (297)$$

So the patient has less than half a percent chance of having the disease despite testing positive for it. The only way to improve this would be to select on the basis of symptoms and/or other tests that increase the prior probability $P(H|I)$ substantially.

Parameter estimation:

Often we need to fit models with free parameters to observed data. This can be treated in Bayesian analysis by considering each value of the parameter being true as an independent hypothesis. From this, one can construct the probability distribution of the parameter value, which will be called the posterior probability distribution.

Let the posterior probability density function of a parameter λ of a model B given data D and context I be written as $p(\lambda|B, D, I)$. By Bayes' theorem

$$p(\lambda|B, D, I) \propto p(D|\lambda, B, I)p(\lambda|I) \quad (298)$$

Note that $P(D|I)$ is independent of λ and is hence common to all values of λ . Here $p(D|\lambda, B, I)$ is the likelihood function. If the so-called "best fit" value of λ is λ_0 , then the likelihood as a function of λ would be maximum at λ_0 . This is then a peaked function with derivative going to zero at maximum. The likelihood function is also positive definite for all values of λ in the relevant domain. One may then define the log-likelihood function $\mathcal{L} = \ln[p(D|\lambda, B, I)]$, which will be a gentler bell shaped curve with a maximum at λ_0 . At the peak,

$$\frac{d\mathcal{L}}{d\lambda} = 0; \quad \frac{d^2\mathcal{L}}{d\lambda^2} < 0$$

Hence, near the peak the function can be written in Taylor series as:

$$\mathcal{L} = \mathcal{L}_0 + \frac{1}{2} \left. \frac{d^2\mathcal{L}}{d\lambda^2} \right|_{\lambda_0} (\lambda - \lambda_0)^2 = \mathcal{L}_0 - \frac{(\lambda - \lambda_0)^2}{2\sigma^2} \quad (299)$$

where

$$\sigma \equiv \left[- \left. \frac{d^2\mathcal{L}}{d\lambda^2} \right|_{\lambda_0} \right]^{-1/2}$$

From the above, we may then write

$$p(D|\lambda, B, I) = p(D|\lambda_0, B, I)e^{(\lambda - \lambda_0)^2/2\sigma^2} \quad (300)$$

which is a Gaussian approximation.

Now let A and B be two models being proposed to explain the same data D . Of these, B has an additional parameter λ by tuning which a higher maximum likelihood than A can be achieved. Which of these two models

is to be preferred? In Bayesian analysis the comparison of models is done via the Posterior ratio

$$r = \frac{P(A|D, I)}{P(B|D, I)} \quad (301)$$

which is also known as the Bayes' factor. In the above example, if $r \gg 1$ then model A is to be preferred and for $r \ll 1$ model B is to be preferred. For $r \sim 1$ the data are not sufficiently discriminatory between the two models.

However, if model B has an additional parameter, then a penalty factor for this must be present in the comparison. Unless the fit improves considerably, a simpler model should still be preferred. This is known as the principle of Occams' razor. Such an Occam factor in fact gets naturally implemented in defining the Bayes' factor.

We can write

$$r = \frac{P(A|D, I)}{P(B|D, I)} = \frac{P(D|A, I)}{P(D|B, I)} \cdot \frac{P(A|I)}{P(B|I)} \quad (302)$$

The first factor is called the *Likelihood Ratio*.

Now since model B has a parameter λ , $P(D|B, I)$ needs to be estimated by marginalisation:

$$P(D|B, I) = \int p(D, \lambda|B, I) d\lambda = \int p(D|\lambda, B, I) p(\lambda|B, I) d\lambda \quad (303)$$

Let us say that a priori λ is expected to lie in the range $\lambda_{\min}$ to $\lambda_{\max}$ with uniform probability. Then

$$p(\lambda|B, I) = \begin{cases} \frac{1}{\lambda_{\max} - \lambda_{\min}} & \text{for } \lambda_{\min} < \lambda < \lambda_{\max} \\ 0 & \text{otherwise} \end{cases} \quad (304)$$

Let λ_0 be the "best fit" value of λ , i.e. $p(D|\lambda, B, I)$ maximises at $\lambda = \lambda_0$. Then, near λ_0 , this can be expressed as in eq. (300). In this approximation,

$$\begin{aligned} P(D|B, I) &= \frac{1}{\lambda_{\max} - \lambda_{\min}} \int_{\lambda_{\min}}^{\lambda_{\max}} p(D|\lambda, B, I) d\lambda \\ &\approx \frac{p(D|\lambda_0, B, I)}{\lambda_{\max} - \lambda_{\min}} \int_{-\infty}^{\infty} e^{(\lambda - \lambda_0)^2 / 2(\delta\lambda)^2} d\lambda \\ &= \frac{p(D|\lambda_0, B, I) \times (\delta\lambda) \sqrt{2\pi}}{\lambda_{\max} - \lambda_{\min}} \end{aligned} \quad (305)$$

where the width σ has been written as $\delta\lambda$, which is assumed to be much less than $|\lambda_{\max} - \lambda_{\min}|$. Then

$$r = \frac{P(A|I)}{P(B|I)} \frac{P(D|A, I)}{p(D|\lambda_0, B, I)} \times \left[\frac{\lambda_{\max} - \lambda_{\min}}{(\delta\lambda) \sqrt{2\pi}} \right] \quad (306)$$

The last factor in the rhs is the Occam factor, which imposes a penalty against additional parameters.

Multi-dimensional problem:

Quite often the model being fit to the data $\{D\} \equiv \{D_1, D_2, \dots, D_M\}$ would have not one but multiple parameters $\{x\} \equiv \{x_1, x_2, \dots, x_n\}$. The Bayesian relation may then be written as:

$$p(\{D\}|\{x\}, I)p(\{x\}|I) = p(\{D\}|I)p(\{x\}|\{D\}, I) \quad (307)$$

which we denote as

$$\mathcal{L}(\{x\}) \times \pi(\{x\}) = Z \times \mathcal{P}(\{x\}) \quad (308)$$

namely, Likelihood $\times$ Prior = Evidence $\times$ Posterior.

Considering that

$$\int \pi(\{x\})d\{x\} = 1; \quad \int \mathcal{P}(\{x\})d\{x\} = 1,$$

we have

$$\int \mathcal{L}(\{x\})\pi(\{x\})d\{x\} = Z \quad (309)$$

and

$$\mathcal{P}(\{x\}) = \frac{\mathcal{L}(\{x\})\pi(\{x\})}{Z} \quad (310)$$

Z will need to be computed numerically from eq. (309). Also in the case of marginalisation:

$$\mathcal{P}(\{x_1, \dots, x_k\}) = \int \mathcal{P}(\{x\})d\{x_{k+1}, \dots, x_n\} \quad (311)$$

and the evaluation of averages, e.g.

$$\langle A(\{x\}) \rangle = \int \mathcal{P}(\{x\})A(\{x\})d\{x\} \quad (312)$$

it is necessary to integrate over the posterior. In case of high dimensionality, the preferred method of sampling the Posterior and carrying out such integrals is the Markov Chain Monte Carlo (MCMC) method.

Monte-Carlo Methods:

Monte-Carlo Methods are often used to generate random samples from a given probability distribution. To do so, the starting point is a uniform random number generator that provides a random value ξ in the range $[0,1]$ with the probability density

$$p(\xi) = \begin{cases} 1 & (0 \leq \xi \leq 1) \\ 0 & \text{otherwise} \end{cases} \quad (313)$$

One of the simplest methods to generate a sequence of such uniform deviates is the Linear Congruential Generator (LCG):

$$I_{j+1} = (aI_j + c) \bmod m \quad (314)$$

$$\xi_{j+1} = (\text{float})I_{j+1}/(\text{float})m \quad (315)$$

where the I -s, a , c and m are integers, with m being a very large value, close to the largest integer representable in the system. Care is required in choosing the combination of a and m to ensure minimum sequential correlation and maximum repeat period. For 32-bit implementation, one good combination is the Park and Miller minimal standard:

$$a = 7^5 = 16807; \quad m = 2^{31} - 1 = 2147483647; \quad c = 0 \quad (316)$$

Direct implementation of this will require storing an intermediate number of more than 32 bits. However that can be avoided by using Scharge's algorithm:

$$\text{If } m = aq + r, \text{ where } q = [m/a] \text{ and } r = m \bmod a, \quad (317)$$

Then for an integer z

$$az \bmod m = \begin{cases} a(z \bmod q) - r[z/q] & \text{if it is } \geq 0, \\ a(z \bmod q) - r[z/q] + m & \text{otherwise} \end{cases} \quad (318)$$

as long as $r < q$ and $0 < z < m - 1$. The square brackets above represent the integer part. For the Park and Miller Minimal Standard Generator $q = 127773$ and $r = 2836$.

An example of a 64-bit generator is MMIX by Donald Knuth:

$$a = 6364136223846793005; \quad m = 2^{64}; \quad c = 1442695040888963407 \quad (319)$$

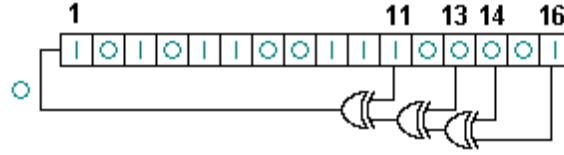


Figure 6: An implementation of a 3-tap 16-bit Linear Feedback Shift Register algorithm (source: Wikipedia)

Another commonly used method of generating pseudo-random sequences is the Linear-Feedback Shift Register (LFSR) algorithm in which a shift register of desired length is first filled with seed bits, not all of which are zero. Some of these bits, including the most significant bit, are then used as taps, which are combined through an exclusive-or algorithm to generate a new bit value which is inserted as the least significant bit and all other bits are shifted upwards. The most significant bit is read off as the output and discarded. The pseudo-random number is constructed by collecting the output bit sequences and normalising the resulting integer. Figure 6 shows a 16-bit example. An m -bit shift register can generate a random sequence of length $2^m - 1$ if the taps are derived from the coefficients of a primitive polynomial of order m .

High-quality random number generators may implement a combination of two distinct families of algorithms, for example the LCG and the LFSR.

Once we have a supply of uniform random variates in the range $[0,1]$, random samples from other distributions can be created using a variety of techniques. The most common among them is the **Inverse Function Method**:

Let the target probability distribution be $p(x)$, defined in the domain $[a, b]$. Then

$$\int_a^b p(x)dx = 1 \quad (320)$$

and its *Cumulative Distribution*

$$F(x) = \int_a^x p(x')dx' \quad (321)$$

Note that $F(x)$ takes values from 0 to 1. If we now choose an uniform

random variate ξ in the range $[0,1]$ and set

$$F(x) = \xi; \quad \text{and hence } x = F^{-1}(\xi). \quad (322)$$

then the resulting x will be distributed according to $p(x)$. This can be seen as follows. Let $\tilde{p}(x)$ represent the distribution of the random samples generated from eq. (322). Then

$$\tilde{p}(x)dx = \tilde{p}(\xi)\frac{d\xi}{dx}dx = 1 \cdot \frac{d\xi}{dx}dx = \frac{dF(x)}{dx}dx = p(x)dx$$

Example: *Exponential distribution:*

$$p(x) = e^{-x}; \quad \int_0^\infty p(x)dx = 1; \quad F(x) = \int_0^x e^{-x'} dx' = 1 - e^{-x}$$

We may then set $\xi = 1 - e^{-x}$, but noting that ξ and $(1 - \xi)$ has the same distribution, we may as well write

$$\xi = e^{-x}; \quad \text{hence } x = -\ln \xi \quad (323)$$

Gaussian Distribution: Box-Muller method:

The target distribution is $p(x) = \exp(-x^2/2)/\sqrt{2\pi}$ for a Gaussian of unit variance. Note that for any other variance σ one needs to just multiply the samples drawn from unit variance by σ .

The cumulative distribution function of this is an error function, which cannot be inverted analytically. The Box-Muller method extends this to two dimensions and uses a transformation between two variable pairs. Consider two quantities x_1, x_2 that are functions of variables ξ_1, ξ_2 . The joint probability distribution transforms between them as

$$p(x_1, x_2)dx_1dx_2 = p(\xi_1, \xi_2)J(\xi_1, \xi_2; x_1, x_2)dx_1dx_2$$

where $J(\xi_1, \xi_2; x_1, x_2)$ is the Jacobian determinant

$$J(\xi_1, \xi_2; x_1, x_2) = \begin{vmatrix} \frac{\partial \xi_1}{\partial x_1} & \frac{\partial \xi_1}{\partial x_2} \\ \frac{\partial \xi_2}{\partial x_1} & \frac{\partial \xi_2}{\partial x_2} \end{vmatrix}$$

Now consider ξ_1, ξ_2 to be two uniform deviates and

$$\begin{aligned} x_1 &= \sqrt{-2 \ln \xi_1} \cos 2\pi \xi_2 \\ x_2 &= \sqrt{-2 \ln \xi_1} \sin 2\pi \xi_2 \end{aligned} \quad (324)$$

the inverse of which is

$$\begin{aligned}\xi_1 &= \exp\left[-(x_1^2 + x_2^2)/2\right] \\ \xi_2 &= \frac{1}{2\pi} \tan^{-1} \frac{x_2}{x_1}\end{aligned}\quad (325)$$

The magnitude of the Jacobian determinant can then be evaluated to yield

$$p(x_1, x_2)dx_1dx_2 = \left[\frac{1}{\sqrt{2\pi}} \exp(-x_1^2/2)dx_1 \right] \left[\frac{1}{\sqrt{2\pi}} \exp(-x_2^2/2)dx_2 \right] \quad (326)$$

So both x_1 and x_2 form independent Gaussian distributions. The task then is to generate two uniform deviates ξ_1, ξ_2 and perform the Box-Muller transformation eq. (324). The resulting deviates x_1 and x_2 will follow a Gaussian distribution of unit variance.

Method of Rejection:

For an irregular, difficult-to-invert distribution $p_1(x)$, one can adopt a technique called the Method of Rejection. Here one defines a smooth analytically invertible function $p_2(x)$ that is everywhere greater than $p_1(x)$. If the total area under $p_2(x)$ is A , and its cumulative function is $F_2(x)$, then

1. A uniform deviate ξ_1 is chosen.
2. From ξ_1 a value of $x = F^{-1}(A\xi_1)$ is derived.
3. A second uniform deviate ξ_2 is generated.
4. If now $\xi_2 < p_1(x)/p_2(x)$, then x is accepted, else it is ignored and the above steps are repeated.

The resulting accepted x values will be distributed according to $p_1(x)$.

Discrete (e.g. integer-valued) variate:

If an intended random variable x can assume only discrete values $x_1, x_2, \dots, x_n$, then its probability distribution can be represented as a stepped histogram. The cumulative distribution of this will be a binned function:

$$F(x) = \begin{cases} \eta_1 & (0 < x \leq x_1) \\ \eta_2 & (x_1 < x \leq x_2) \\ \dots & \\ 1 & (x_{n-1} < x \leq x_n) \end{cases}$$

One can then generate a uniform deviate ξ , and derive x as

$$x = \begin{cases} x_1 & (0 < \xi \leq \eta_1) \\ x_2 & (\eta_1 < \xi \leq \eta_2) \\ \dots & \\ x_n & (\eta_{n-1} < \xi \leq 1) \end{cases} \quad (327)$$

Method of superposition:

If a probability distribution is composed of a weighted sum of multiple distributions, say

$$p(x) = \sum_m C_m p_m(x) \quad (328)$$

where

$$\sum_m C_m = 1 \quad \text{and} \quad \int p_m(x) dx = 1$$

then it can be simulated as follows:

1. Choose a uniform deviate ξ_1
2. Using ξ_1 , choose m from a discrete distribution $p(m) = C_m$
3. Choose a second uniform deviate ξ_2
4. Generate $x = F_m^{-1}(\xi_2)$ where $F_m(x) = \int_{-\infty}^x p_m(x') dx'$

An example of this is the photon number density as a function of photon energy in blackbody radiation:

$$p(E) = \frac{1}{2\zeta(3)} \beta^3 E^2 (e^{\beta E} - 1)^{-1} \quad (329)$$

$$= \frac{1}{\zeta(3)} \sum_{m=1}^{\infty} \frac{1}{m^3} p_m(E) \quad (330)$$

where

$$p_m(E) = \frac{m^3}{2} \beta^3 E^2 e^{-\beta m E} \quad (331)$$

Estimators:

The purpose of a Monte-Carlo method is to evaluate desired estimators, which are averages of intended functions over the target probability distribution. Let the random variable $\eta = g(\xi_1, \xi_2, \dots)$ be distributed according to the probability distribution $p(\eta)$. Let a be the true value of the expectation value of a function $f(\eta)$, i.e.:

$$\langle f(\eta) \rangle = \int f(\eta)p(\eta)d\eta = a$$

The Monte-Carlo estimator of a using n samples of η is

$$a_n = \frac{1}{n} \sum_{i=1}^n f(\eta_i)$$

a_n will tend to a as $n \rightarrow \infty$, but for a finite value of n there is an error on the estimator, the variance being

$$\sigma_n^2 = \langle f(\eta)^2 \rangle_n - a^2$$

and the probable error on a is

$$\sigma_a = \sqrt{\frac{\sigma_n^2}{n}}$$

The result converges slowly, as the Monte-Carlo error drops only as $1/\sqrt{n}$.

It is therefore desirable to design the Monte Carlo algorithm in a fashion that makes σ_n^2 as small as possible.

Let

$$\langle f(\eta) \rangle = \int f(x)p_\eta(x)dx = a$$

For which

$$a_{n,\eta} = \frac{1}{n} \sum_{i=1}^n f(\eta_i); \quad \sigma_\eta^2 = \langle f^2(\eta) \rangle - a^2$$

If we now introduce another distribution $p_\zeta(x)$:

$$\begin{aligned} \langle f(\zeta) \rangle &= \int f(x) \frac{p_\eta(x)}{p_\zeta(x)} p_\zeta(x) dx = a \\ &\equiv \int f(x) w(x) p_\zeta(x) dx \end{aligned} \tag{332}$$

for which

$$a_{n,\zeta} = \frac{1}{n} \sum_{i=1}^n f(\zeta_i) w(\zeta_i); \quad \sigma_{\zeta}^2 = \langle f^2(\zeta) w^2(\zeta) \rangle - a^2$$

We would have $\sigma_{\zeta,n}^2 < \sigma_{\eta,n}^2$ if $w(x) < 1$

As both p_{η} and p_{ζ} are normalised probability distributions, $w(x)$ cannot be < 1 for all x . However, significant reduction in Monte-Carlo noise is possible if $w(x) < 1$ over the region where $f(x)$ is large. This is known as *importance sampling*.

This applies also to Monte-Carlo integration methods. If $f(\vec{x})$ is the quantity being integrated in multi-dimensions, then the Monte-Carlo variance in the integral can be reduced by introducing a sampling function $p(\vec{x})$. The variance is minimised when $p(\vec{x})$ is proportional to $f(\vec{x})$.

Generating random samples of a multi-dimensional complex probability distribution can be a challenge. One popular approach is the Markov Chain Monte Carlo technique.

Markov Chain Monte-Carlo:

The concept of a Markov chain derives from the description of stochastic processes. We will consider here a discrete time process, where a random variable Z takes values $\{z_1, z_2, \dots, z_i, \dots\}$ at times $\{t_1, t_2, \dots, t_i, \dots\}$. The following probabilities may then be defined:

- $\rho_1(z, t)$: probability that state z is present at time t
- $\rho_2(z_2, t_2 | z_1, t_1)$: conditional probability that state z_2 is present at time t_2 given that state z_1 was present at time t_1
- ⋮
- $\rho_n(z_n, t_n | z_{n-1}, t_{n-1}; z_{n-2}, t_{n-2}; \dots; z_2, t_2; z_1, t_1)$
- ⋮

In general all of these probabilities need to be specified in order to fully describe a random process. A very special process arises when

$$\rho_n(z_n, t_n | z_{n-1}, t_{n-1}; z_{n-2}, t_{n-2}; \dots; z_2, t_2; z_1, t_1) = \rho_2(z_n, t_n | z_{n-1}, t_{n-1}),$$

namely that the probability of a state depends only on one previous state. Such a process is called a *Markov Process*. The probability ρ_2 , called the *Transition Probability*, obeys

$$\rho_2(z_3, t_3 | z_1, t_1) = \int dz_2 \rho_2(z_3, t_3 | z_2, t_2) \rho_2(z_2, t_2 | z_1, t_1) \quad (333)$$

$$\rho_1(z_3, t_3) = \int dz_2 \rho_2(z_3, t_3 | z_2, t_2) \rho_1(z_2, t_2) \quad (334)$$

So $\rho_1(z, t_0)$ and ρ_2 determine the course of evolution ($t_0 \equiv$ start time).

While $\rho_1(z, t)$ would in general evolve with time, if the process is one with a stationary limit, then it will converge to a time independent stationary distribution as large times:

$$\rho_{\text{stat}}(z) = \lim_{t \rightarrow \infty} \rho_1(z, t)$$

For a system with discrete states n

$$\rho_{\text{stat},n} = \lim_{t \rightarrow \infty} \rho_n(t) \equiv \rho_n^*$$

Now the rate of change of $\rho_n(t)$ may be written as

$$\dot{\rho}_n(t) = \sum_{n' \neq n} (w_{nn'}(t)\rho_{n'}(t) - w_{n'n}(t)\rho_n(t))$$

where $w_{nn'}$ is the transition probability from state n' to n and vice versa.

A stationary state is reached when $\dot{\rho}_n(t) = 0$. Thus

$$\sum_{n' \neq n} w_{nn'} \rho_{n'}^* = \sum_{n' \neq n} w_{n'n} \rho_n^* \quad (335)$$

Since $\dot{\rho}_n(t) = 0$ for every single state n , a detailed balance prevails:

$$w_{nn'} \rho_{n'}^* = w_{n'n} \rho_n^*$$

and hence

$$\frac{w_{n'n}}{w_{nn'}} = \frac{\rho_{n'}^*}{\rho_n^*} \quad (336)$$

The above can be used to define a progression from one state to the next. If a stationary process exists, then regardless of the initial state, after sufficient number of steps, the process will converge to ρ^* , provided the right transition probability is specified.

Let us say that the target probability distribution is $p(\vec{x})$. We treat this as the stationary distribution of a Markov process, thus specifying $\rho^* = p(\vec{x})$.

One then starts with an initial state and evolves according to the transition probabilities appropriate to the target distribution. Let $\vec{x}_n$ be the current state and $\vec{x}_{n'}$ be a proposed next step. We have $\rho_n^* = p(\vec{x}_n)$, $\rho_{n'}^* = p(\vec{x}_{n'})$ and

$$\frac{w_{n'n}}{w_{nn'}} = \frac{\rho_{n'}^*}{\rho_n^*} \equiv \alpha$$

We then use a stepping probability

$$W_{n'n} = \begin{cases} \alpha & \text{if } \alpha < 1 \\ 1 & \text{if } \alpha \geq 1 \end{cases}$$

Namely, if $\alpha \geq 1$ then we accept the next state and move to it as the next element in the sequence. If not, then we generate a uniform deviate ξ and move to the new state if $\xi < \alpha$, else reject the proposal and generate a fresh proposal for the state n' . This is known as the Metropolis-Hastings algorithm.

The sequence of states generated in this process is referred to as the Markov Chain. The states in the chain would be distributed according to the target probability distribution as $n \rightarrow \infty$. In practice the states traversed in an initial portion of the chain called the *burn-in* length are discarded, and the rest are accepted as members of the target distribution. The burn-in length may be decided based on how long it takes for the value of a chosen estimator to stabilise.

The efficiency of acceptance may be improved by a suitable choice of the proposal distribution $Q(\vec{x}'|\vec{x})$. A closer resemblance of this to the target distribution enhances the efficiency. Often, however, a Gaussian centred at $\vec{x}$ is employed. Care must be taken, though, to not get confined to a local maximum in the case of a multi-peaked distribution.

Adaptive Rejection Metropolis Sampling

This works by first generating states within an envelope distribution, and then using those states in Metropolis-Hastings steps. Let $p(x)$ be the target distribution and $g(x)$ an envelope distribution that is *log-concave*. One then defines $h(x) = \log g(x)$. Instead of $h(x)$ itself, one may now use a piecewise linear approximation. The first step would be to just draw tangents at both ends of the distribution and use the triangle formed by these two as the approximate envelope distribution $E(x)$. A sample η is then drawn from $E(x)$ via inverse function method. Drawing another uniform random number ξ , one accepts $\eta = x$ if $\xi < h(\eta)/E(\eta)$. Once accepted, $E(x)$ is updated by adding $h(x)$ as a point and the tangent to $h(x)$ at x as a new segment of the piecewise linear function (fig. 7). The accepted values of x are then used to run the Metropolis-Hastings generator for $p(x)$.

Gibbs Sampling

The Gibbs Sampling method is particularly suited to situations where conditional probabilities are known, for example in Bayesian analysis.

Here, to sample a posterior $p(\vec{X})$, where $\vec{X} = \{x_1, x_2, x_3, \dots, x_n\}$, one makes use of conditional probabilities $p(x_1|x_2, x_3, \dots, x_n)$ etc.

One starts with a state $\vec{X}^{(0)}$ and steps through subsequent steps $\vec{X}^{(1)}, \vec{X}^{(2)}, \dots, \vec{X}^{(i)}, \dots$

From

$$\vec{X}^{(i)} = \{x_1^{(i)}, x_2^{(i)}, x_3^{(i)}, \dots, x_{j-1}^{(i)}, x_j^{(i)}, x_{j+1}^{(i)}, \dots, x_n^{(i)}\}$$

the next step is taken as follows. To find the components of $\vec{X}^{(i+1)}$, each com-

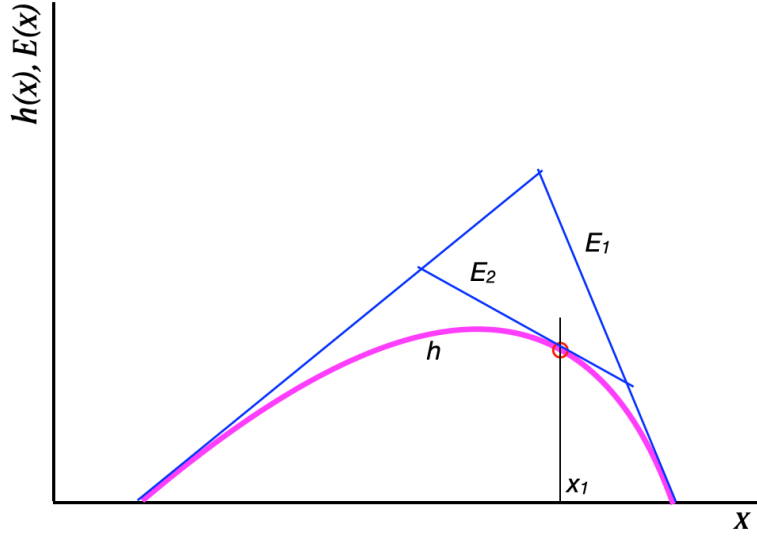


Figure 7: Successive improvement of the piecewise linear approximation of the envelope function $E(x)$ in Adaptive Rejection Metropolis Sampling. $h(x)$ is the original log-concave envelope distribution. The first approximation here is $E_1(x)$, made up of two tangents at the extremities of $h(x)$. x_1 is the first random sample accepted thereafter. $E(x)$ is then updated to $E_2(x)$ by adding a new segment that is a tangent to $h(x)$ at $x = x_1$.

ponent is sampled independently and sequentially from the corresponding conditional probability distributions. Namely, $x_j^{(i+1)}$ is to be chosen from

$$p(x_j | x_1^{(i+1)}, x_2^{(i+1)}, \dots, x_{j-1}^{(i+1)}, x_{j+1}^{(i)}, x_{j+2}^{(i)}, \dots, x_n^{(i)})$$

This is to be repeated for all $j = 1 \dots n$ to construct the next state $\vec{X}^{(i+1)}$.

If the conditional probability distributions employed above correspond to the true posterior then the generated state can be directly accepted. If, instead, they are approximants then the new state may be subjected to Metropolis-Hastings rule before being accepted.

The following may be noted:

$$p(x_j | x_1, x_2, \dots, x_{j-1}, x_{j+1}, \dots, x_n) = \frac{p(x_1, x_2, \dots, x_n)}{p(x_1, x_2, \dots, x_{j-1}, x_{j+1}, \dots, x_n)}$$

On the rhs, the numerator is the full posterior and the denominator is the posterior marginalised over x_j . As a result, the denominator is independent of x_j and acts only as a normalisation factor. If one employs a sampling method in which the normalisation is not important then the posterior can be used directly.